JAVA

Introduction To JAVA

Why **JAVA**?: It solves some main problems of c, c++.

1] **SIMPLICITY**: JAVA c, c++ se zayada simple thi.

2] **SECURITY**: JAVA c, c++ se zayada secure thi.**JVM** me **sandbox** model hota ha jo code ko secured environment deta ha.

3] **PORTABILITY**:c or c++ platform dependent languages ha, inka jo code compile hota tha vo sirf kisi particular platform k lia hi hota tha.

EX: compiled code 10100101 bnaya ha platform A (Processor+OS) ke lia, to ye code platform B(Processor+OS) ke lia work nahi krega.

HOW DID JAVA SOLVE THIS PORTABILITY PROBLEM?

JAVA took the [concept of **BYTECODE**]

EXAMPLE:

C++ me hume dono platform k lia alag-2 compile krna pdta tha.

C++ code [Hello.cpp] =>compiler =>machine code [010…..]=> platform A
C++ code [Hello.cpp] =>compiler =>machine code [110…..]=> platform B

COMPILER AND INTERPRETER: Dono ka kaam same hi ha [Source code ko machine code me change krna].

Compiler: convert code all at once, fir sara converted code ek sath run krta ha. Example: **c, c++**

Interpreter: Convert code line by line,or har converted code ko sath ke sath run bhi krta ha. Example: **Python.**

JAVA is both language: Kuki ye Interperter or Just In Time [JIT] Compiler dono ka use :krta ha.

JAVA code [Hello.java]=>compiler=>**BYTECODE** [Hello.class]=>**JVM**=>**INTERPRETER + [JIT] COMPILER** =>machine code [110…..]=> Platform A,B,C,D…..

JVM, JRE, JDK

JVM [Java Virtual Machine] =>**[0]** JVM bytecode ko securely machine code me badalta ha interpreter or JIT compiler ka use krke.

[1] JAVA is called Platform Independent. **WORA** [Write once, Run anywhere].

[2] JVM me **sandbox** model hota ha jo BYTECODE ko secured environment deta ha.

[3] Garbage collection.

JRE [Java Runtime Enviroment]=> **JVM + CLASS LIBRARIES**

JDK [Java Development Kit] => **JRE + COMPILER + DEBUGGER + JAVA DOCS.**

JSE, JEE, JME

JSE [**Java Standard Edition]**: Core Java. basic java k lia.

JEE [**Java Enterprise Edition**]: ye thodi advance ha isme or methods h java ko web apps pr kaam krne k lia. Also known as JAKARTA EE.

JME [**Java Micro Edition]:** YE phle use hota tha ab nhi hota.

VARIABLES

VARIABLES: ek tarah se memory me container ki tarah hota ha jo kisi value ko hold krte ha.

Variables hamesha **[DATATYPE + IDENTIFIERS =** [Value] **LITERAL + (;)**] ko mila k bnte ha.

IDENTIFIERS

IDENTIFIER: ka container me kisi bhi value ko represent krne ka kaam hota ha.

DATATYPE

DATATYPE: Iska kaam ha ye batana ki ek **VARIABLE** me kis type ka **IDENTIFIER** ha.

2 TYPES OF DATATYPE

[1] PRIMITIVE DATATYPES	**[2] NON-PRIMITIVE DATATYPES**
A] Integer : byte, short, int, long. **B] Real no :** float, double **C] Character:** char **D] Boolean:** boolean	

A] Integer : byte, short, int, long.

INTEGER	WIDTH IN BITS	RANGE
Byte	8 [_ _ _ _ _ _ _ _] **[2^8-1] = 255** Iska mtlab 8 bit total 255 no carry kr skti ha.	-128 to 127 [total=255 no_s ha]
Short	16 **[2^n-1] = total numbers**	-32,768 to 32767
Int	32	-2,147,483,648 to 2,147,483,648
Long	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

B] Real no : float, double. [**Ex:** 5.23, 31.22] in decimal no's ko computer language me **Floating Point Numbers** khte ha.

REAL NO	WIDTH IN BITS	APPROX.RANGE
Float=single precision	32	1.4e-045 to 3.4e+038
Double=double precision	64	4.9e-324 to 1.8e+308

C] Character: [char] can represent any **{ UNICODE CHARACTER }**=ye har language ke symbol, alphabet, sab k lia **INTEGER** value deta ha.

CHARCTER	WIDTH IN BITS	APPROX.RANGE
Char	16	**Unicode** character

D] Boolean: boolean KE sirf 2 values hoti ha **TRUE** or **FALSE.**

KEYWORDS

Ye ase words hote ha jo java ne apne liye reserve krke rkhe ha or unhe hum use ni krr skte.
Ex: class, public, static, void.

HOW NEGATIVE NUMBERS ARE STORED IN MEMORY

Integers ke negative no's ko store krne se shuru krte ha.
Example: **byte b = - 42 ;**

STEP-1] Sabse phle hum **Positive 42** ko binary m likhege jo hota ha **[00101010]** or positive value ase hi memory me store hoti ha. Or isko nikalne ki trick ha ki hume 42 ka **LCM** lena hoga or fir uske jo remainders ase hoge [101010] bs inko once place se lagao or jo space bche usme **0's** lga do jitni bits ka space bcha ha. Jse byte me abhi 2 space bche ha.

STEP-2] JAVA me negative no's ko save krne ke tarika ko [**2's COMPLIMENT**] khte ha.

1st COMPLIMENT : Isme hum particular no ke binary bits ko reverse kr dete ha.Matlab hum 1 ko 0 me, or 0 ko 1 me badal dege.
To 42 ke binary bit = [00101010]----------->[11010101] This is our 1st Compliment.

2nd COMPLIMENT : Isko nikalne ke lia muj bs 1st Compliment me **1 ADD** krna ha.
To fir 11010101 + 1 = **11010110** **, hence Ye ha hmari final binary value jo memory me store hogi.**

Ek **important** baat jabhum memory se value ko reterive krege to kse pta chlega ki **11010101** value **+ve** no ki ha ya **-ve** no ki ? yha pr ate ha **MSB** and **LSB.**

MSB [Most Significant Bit]: MSB kisi bhi number ki left most side hoti ha, or iski significant value zada hoti ha.
Ex: 259874, to yha pr 2 MSB ha.

LSB [Least Significant Bit]: LSB kisi bhi number ki right most side hoti ha, or iski significant value km hoti ha.
Ex: 259874, to yha pr 4 LSB ha.

To CONCLUSION ye ha agar kisi number ki **MSB** value **1** ha to vo **-ve** no ha, or agar **MSB** value **0** ha to o **+ve** no ha.

HOW REAL NUMBERS ARE STORED IN MEMORY

FLOATING NUMBERS

Total Bits=32, they are divided in **3** parts.

1Bit=**Sign** Bit, **8** Bits=**Exponent**, **23**Bits= **Mantissa**

float f = 8.125f ;

SAVING TO MEMORY

STEP 1)
firstly convert **8** to direct binary language=**1000** by doing LCM.

STEP 2)
Now, convert **0.125** to binary= Iske liye hume 0.125 ko 2 se multiply krna pdega,
So, 0.125 * 2 = 0.25
0.25 * 2 = 0.5
0.5 * 2 = 1.0
∴ iska binary code bna **001**, teeno result ke 1st digits.
Now binary value of 8.125 is **1000.001**

STEP 3)
Make it in form of **(1.xx)*2^{exponent}=> 1.000001*2^{3}**
∴ humne 1000.001 ko 1.xx me convert krne k lia decimal ko three digit aage laya.=> 1.000001 or decimal jitne digit aage aya utni value +sign ke sath 2 ke exp. Me jud jaygi.

STEP 4)
Add **BIAS** to the exponent=3
BIAS value=127
=> 3+127=**130**
Now, convert 130 into binary value by doing LCM.=**10000010**, ye value 8 BIT **exponent** me aayegi.

STEP 5)
Place values in **MEMORY**=Total=32 bits

1 Sign bit = 0 [kuki 8.125 +ve value ha]
8 Exponent = 10000010 [ye step-4 se mili ha.]
23 Mantissa = 000001_00000000000000000 [ye value xx wali ha jo hume Step-3 se mili ha] or jo blank spaces ha usme 0's lg jaygi jab tak 23 bits complete nhi hogi

RETERIVE FROM MEMORY

Print Statement=> System.out.println(f);

Formula**=(-1)$^{SIGN\ BIT}$ * (1 + Mentissa) * 2$^{exponent\ -\ BIAS}$**

=> $(-1)^0$ * (1 + Mentissa) * $2^{130-127}$
= 0 * (1 + Mentissa) * 2^3
=> **mentissa =2^{-6}** kuki mentissa decimal ke baad ki value ha to isko 2^{-1}, 2^{-2}, 2^3…… me left to right count krege jaha-jaha hume '1' milega. To yha hmari value ati ha 2^{-6}
= 0 * (1 + 2^{-6}) * 8, or isko solve krke aa jaygi value= **8.125** ans.

BIAS

Float vaulue ko memory me store krna ka jo 32bit format ha ye **IEEE STANDARD** ke hisab se ha.
Or **IEEE** ne **BIAS** ko sirf is liye banaya taki koi bhi **exponent** value **-ve** na rhe.
Formula=>**($2^{(exponent-1)}$-1)** => 2^{8-1}-1 = 2^7-1 = 128-1 = **127,BIAS value.**

HOW DOUBLE NUMBERS ARE STORED IN MEMORY

There is a same concept with **double** numbers.
Bs double me bits **64** ho jati ha or exponent bhii **11** ho jate ha.
Baki sara process same hi rhta ha.
Or double me **BIAS 1023** hota ha.

Type Conversion In Java

Type conversion ka matlab hota ha ki hum kisi **datatype** ko ek-dusre me convert kr sakte ha. **Ex:** Group **integer** me hum **byte** ko **short, int, long** me convert kr skte ha.

Types of "Type Conversion In Java"

1. **Implicit:** Jo type conversion java khud se kr de. [Small bit datatype to large bit datatype]
   Example: **byte b = 24; ⇒ int i = _; ⇒ i = b; ⇒ system.out.println (i); = 24** print ho jyga, and this conversion is known as **WIDENING CONVERSION.**
   ∴, RULE FOR IMPLICIT CONVERSION ⇒
   Destination datatype should be wider (more bits) then source datatype.

2. **Explicit:** Jo type conversion hume manually krna pde **CASTING** ke through. [large datatype to small datatype].
   Example: **int ii = 300; ⇒ byte b = _; ⇒ b= ii;** ❌
   Int ii = 300; ⇒ byte b = _; ⇒ b = (byte) ii; ✅ **⇒ system.out.println (b);** = kuche ye process hoga. (Step 1) ii = 300, binary me convert ho jyga = **{100101100}**
   Pr 300 int value ha to iski range 32bits ha to iski age ki values me 0's lg jyge.

8 bits	8 bits	8 bits	8 bits
00000000	00000000	00000001	00101100

   Ab byte be ki value last wala **cell** hoge kuki byte ki value 8 bits ki hoti ha.
   ∴ byte b = 00101100 ⇒ 44 ans,
   And this conversion is known as **NARROWING CONVERSION.**

```
int ii = 300;
byte bb;
bb = (byte)ii;
System.out.println("case 2 of explicit [byte to int]-->"+ bb);
```

   Result: case 2 of explicit [byte to int]-->44

Automatic Type Promotions Conversion

Isme java compiler automatically small datatype ko bade datatype me convert kr deta ha, taki data loss na ho.

Types of Automatic Type Promotion Conversion

1. Byte, short and char values are promoted to int.
2. If 1 operend is long, then whole expression will become long.
3. If 1 operend is float, then whole expression will become float.
4. If 1 operend is double, then whole expression will become double.

```
//Type promotions
// agar ab hm asa kre byte b = 50; --> b = b*2; to kya ye possible ha?
//or isme type mismatch wala error ku aa ra ha int to byte conversion wala iska solu notes
me ha.
```

```
byte bpp = 50;
bpp = (byte)(bpp*2); // ab result 100 ana chahiye
System.out.println("Automatic Type promotion-->"+bpp);

// Applying all four rules of Automatic Type Promotions here is code below
byte bbb = 42;
char ccc = 'a';
short sss = 1024;
int iii = 50000;
float fff= 5.67f;
double ddd = .1234;
// yha  pr dekho sabhi datatypes me double datatype sabse bda ha to sara result double me hi
calculate hoga.
// isi liya niche jo bhi calculatons ha usme hum double datatype ka use kr rhe ha.
double result = (fff * bbb) + (iii / ccc) - (ddd * sss);
/*ab yha pr bhi  har category me result bde datatype me hi niklega.
1) fff * bbb = float
2) iii / ccc = int
3) ddd * sss = double
therefore, float + int = float - double =  double        */
System.out.println((fff*bbb) + " + " + (iii / ccc) + " - " + (ddd * sss));
System.out.println("result="+ result);
```

Result automatic Type promotion-->100
238.14 + 515 - 126.3616
result=626.7784146484375

Operators in JAVA

Arithmmetic Operators:

1. **Basic operators⇒ (+, -, *, /, %)**

```
int a = 5;
 int b = 10;
// Now, performing Basic Arithmatic expressions.
 int c = a + b;  //15
 int d = a - b;  //-5
 int e = a * b;  //50
 int f = b / a;  //2
 int g = b % a;  //0
System.out.println( "sol of c, d, e, f, g, --> " + c + " , " + d + " , " + e + " , " + f + "
, " + g);
 int h = a + 2; // a = 5 therefore 5+2=7 ==> h = 7
 System.out.println(" h = a + 2 --> " + h);
// ab agar muj "h" ki value me "+2" krna ha to me direct bina kisi datatype ke kr skta hu
h = h + 2; // h = 9
System.out.println("h + 2 --> "+ h);
```

2. Compound assignment operators ⇒ +=, -=, *=, /=, %=

```
// Additional assignment operator.
h += 3; // NOTE---->  short trick [h = h + 3]--->[h += 3] dono ka same mtlb ha.
System.out.println("h += 3 --> "+ h);


// subtraction assignment operator.
h -= 1;
System.out.println("h -= 1 --> "+ h);

// multiplication assignment operator.
h *= 2;
System.out.println("h *= 1 --> "+ h);
```

3. Encriment and decriment operators ⇒ ++, - -

```
//Increment operators
int i = 6; // lets take an example of this.
System.out.println("int i = 6 --> " + i);


// post increment operator (j++)
i++; // yha pr bhi iska mtlab ha (i = i + 1) or jo second wala '+' ha uska mtlab ha
'i' ko '1'se badana.
System.out.println("Increment (1++) --> " + i);


// Post decriment operator (j--)
i--; // it means (i = i-1). Jo yha second '-' ha uska mtlab ha value me '1' subtract
krna.
System.out.println("Post decriment operator (1--) --> " + i);


// Pre increment operator ( ++j )
int j = 8;
int k = j++; //This is post increment operator, it means 'k' ko 'j' ki value do or 'j'
me '1' add kr do.
System.out.println("(k = j++) ---> " + k + " , " + j); // ans--> k = 8, j = 9
k = ++j; // it means phle 'j' me '1' add kro fir jo value aaye wo 'k' ki value hogi.
System.out.println("(k = ++j) ---> " + k + " , " + j); // ans --> k = 10, j = 10


// Pre decriment operator ( --j )
int l = 14;
l = --j; // it means phle 'j' se '-1' kro fir us value ko 'l' me dal do.
System.out.println("(l = --j) ---> " + l + " , " + j); //yha ans,---> l = 9, j = 9
```

Relational Operators

Types of Relational Operators: **(==, !=, <, >, <=, >=)** ye hamesha **BOOLEAN** Datatype ke sath hi kam krte ha.

Bhot simple concept ha,

Lets take two statements, int a = 5;
Int b = 10;

If our statement is,

a = b (it means ki **b** ki jo bhi value ha use **a** me assign kr do.), this is known as **assignment operator.**

And, if our statement is,

1. **a == b)** is **a** equal to **b**
2. **a != b)** Is **a** not equal to **b**
3. **a < b)** is **a** smaller then **b**
4. **a > b)** is **a** greater then **b**
5. **a >= b)** is **a** greater then or equal to **b**
6. **a < = b)** is **a** less then or equal to **b**

```
//Relational Operators (==, !=, <, >, <=, >=), always works with boolean datatype.
int a  = 5;
int b = 10;

boolean c = (a==b);
System.out.println("boolean c --> " + c); // Result = false
boolean d = (a!=b);
System.out.println("boolean d --> " + d); // result = true
boolean e = (a<b);
System.out.println("boolean e --> " + e); // result = true
boolean f = (a>b);
System.out.println("boolean f --> " + f); // result = false
boolean g = (a<=b);
System.out.println("boolean g --> " + g);// result = true
boolean h = (a>=b);
System.out.println("boolean h -->" + h); // reult = false
```

Bitwise Operators

Bitwise Operators: ye hume bit manipulation krne me help krte ha.
Bit manipulation: it means ki ye (0) or (1) ke level pr jaa ke kaam krte ha.

LIST OF BITWISE OPERATORS

1. **'&' Bitwise and**
2. **'|' Bitwise or**
3. **'^' Bitwise exclusive or**
4. **'~' Bitwise not**

Lets take an example: byte a = 2;
byte b = 3;
∘˚, bit value of, a = 00000010
B = 00000011

Truth Table					
A	**B**	**A & B** (both 1 chahiye)	**A | B** (ek 1 bhi chlega)	**A ^ B** (odd no of 1 chiye)	**~A** (reverse of a)
0	**0**	**0**	**0**	**0**	**1**
0	**1**	**0**	**1**	**1**	**1**
1	**0**	**0**	**1**	**1**	**0**
1	**1**	**1**	**1**	**0**	**0**

5. **'<<' shift left:** Isme no left shift hoga or values double hone lagti ha.
6.

Example: byte b = 7; (to yaha pr sabse phle byte hamara type promote hoga int me.)
⇒ b = << 1 (to hamari 7 ki sabhi binary values 1-1 step left side shift ho jaygi.)
Or agar ab hm krte ha b = << 34 to vo mode ho jayga 32 se . or value aaygi 2 to hamri values 2 steps shift ho jaygi.

7. **'>>' shift right**
8. **'>>>' shift right with 0 fill**

Logical Operators

Ye boolean ke sath use hote ha jase ki Relational operators hote the.

1. **&&** (and logical operator)
2. **||** (or logical operator)
3. **!** (not logical operator)

Yha pr 1 or concept ata ha **SHORT-CIRCUIT**

FOR example:
```
int a = 5;
int b = 10;
int c = 15;

boolean d = (a < b) && (a < c);
```

Yha pr hume pta ha ki && ka lia hume dono conditions true chiye or agar kisi tarah se hme phle hii pta chle ki hamri ek value false ha to hume dusri value nikalne ki zarurat hi ni ha.

```
int a = 5;
int b = 10;
int c = 15;

boolean e = (a > b) || (a < c);
```

Yha bhi samee ham jante ha ki || ke lia sirf ek hi value true chaiye, or agar kisi tarah pta chle ki meri ek value true ha to dusri value automatically true ho jaygi

Assignment Operators

```
// "=" assignment operator ha or "==" relational operator ha.

int a = 10;
int b = 10;
int c = 10;

// in assignment operator we can do, but we cant do with relational operators
  a = b = c = 10;
System.out.println(a + b + c - a);
```

Ternary Operators.

Ye operators three operands ke bich me lagta ha.

Operator precedence

Isko hum operators ka BODMAS bolege, ya batata ha ki agar bhot operators use ho re ha t phle kon operate krega or last me kon operate krega.

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ~ ! (type)	Right to left	Unary prefix
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
<< >> >>>	Left to right	Shift
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
|	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
||	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

Flow of Control

Flow of Control simply means ki koi bhi code :statemnts ko jab compile krte ha to vo ek ordered flow me compile hota ha, example uper se niche ko

Types of flow of control (FWP)		
Selection FWP	**Itration FWP**	**Jumps FWP**
1. If **2. If-else** **3. If-else-if** **4. Nested-if** **5. switch**	**1) Loops:** **a) While loop** **b) Do-while loop** **c) For loop**	**1. Break;** **2. continue;**

IF: Iska simpaly mtlab ha ki apki statement k lia do raste ha.

If-else: Iska matlab ha ki ya ap 1st path chose kro 2nd choose.

If-else-if: if ke baad else me fir se if lagana.

Nested-if: iska matlab ha ki if ke andar or if lagana.

Switch:

- lagbhag if-else-if ki tarah same ha. Bs ye in se kaafi optimized ha.
- It can be only evaluate to **byte, short, int, char, enumeration.**
- It can't be used with **boolean** values.
- No duplicate **case** is allowed.
- After jdk7 strings can also be used as switch expression.
- After jdk14 switch is enhanced too much.

Switch	**if-else-if**
1. Can only test equality. (int i = 4;) **2. It is more efficient and faster** **3. Ye jump table banata ha jisme ye direct particular case pr chala jata ha.** **4.**	**1. It can test equality as well as inequality. (boolean b = b<3;)**

Itration FWP
2) Loops: **a) While loop** **b) Do-while loop** **c) For loop**

Arrays

Arrays: ye bs collections of datatype ha.
Or isme hum data continuos memory me store krte ha.
Define Arrays: int [ ] rollnumber; / int rollnumber[ ] ;
Determine Arrays: int [ ] rollnumber = new int [50];

Dimensions of Arrays

1 Dimension Array: Simple array ha jisme hum particular datatype ko memory ke particular block me store kr sakte ha.
Example:
int [ ] rollnumber = new int [50];
R1= 101
R2= 102
R3= 103
Sol:

```
// Arrays 101, 102, 103
        int[] rollnumbers = new int [50]; //to determine new array "[]"
        int rollnumbers[] = new int [50];// isko hum ase bhi likh skte ha.

        System.out.println(rollnumbers.length); //ye arrays ka size batata ha, ek continus memory me.
        int x = 101;
        for(int i = 1; i <= rollnumbers.length; i++){
            System.out.println(x);
            x++;
        }
```

2 Dimension Array: **Isme hum array ke andeer array dalte ha. Isliye isko Array of Arrays bhi khte ha.**

```
2 dimension Array ---> each column with different length:
            int marks[][] = new int[3][]; // iska matlab ha ki har 'row' ke columns manualy daluga.
            marks[0] = new int [1]; // yaha pr '0th' row ha usme '1-column' ha.
            marks[1] = new int [2];// yaha pr '1st' row ha usme '2-columns' ha.
            marks[2] = new int [3];// yaha  pr '2nd' row ha usme '3-columns' ha.

            marks [0][0] = 23;

            marks [1][0] = 24;
            marks [1][1] = 90;

            marks [2][0] = 35;
            marks [2][1] = 53;
            marks [2][2] = 89;
            for (int row = 0; row < marks.length; row++){
                for (int column = 0; column < marks[row].length; column++){
                    System.out.print(marks[row][column] + " ");
                }System.out.println();
            }
```

Agar muj arrays ki value pta ha to me isko ase bhhi likh sakta hu.

```
// uper wale array ko me aasse bhi represent kr sakta hu.

           int marks[][] = {
               {23},
               {24, 90},
               {35, 53},
           };
                       for (int row = 0; row < marks.length; row++){
                           for (int column = 0; column < marks[row].length; column++){
                               System.out.print(marks[row][column] + " ");
                           }System.out.println();
                       }
```

How Arrays really work ?

Jab bhi hum koi bhi primitive datatype(byte, short, int, long, double, char, boolean, etc) ko store krtte ha to wo directly stack me store hote ha.

Orr

Vahi jab bhi hum non- primitive datatype (string “”, array[ ]) ko store krte ha to vo HEAP memory me store hote ha. Or isko hum ‘new’ keyword se bnte ha.

Functions

A function is a self- contained block of code designed to perform a specific task. It takes inputs, processes them. And Optionaly returns an Output.

```
public static void main(String[] args) {
        // Functions in Java (Function hamesha PSVM ke bahar banega.)
          //1st call
           greet();

           //2nd call
           sayHello("Priyansh"); // yaha pr (priyansh) ek Argument ha.
           //3rd call
           int x = getNumber();
           System.out.println(getNumber());
           //4th call
           multiply(12,3);
           System.out.println(multiply(12,3));



    }
       /* Type of Functions---> there are "4" type of functions.
          Type-1) No input, No output.                    */
           static void greet() { // jab bhi {no in, no op} hota ha too vha pr {Return type}-->
"VOID" lagate ha.
               System.out.println("Hello");
              // return; // ye bhi optional ha.
           }
       // Type-2) Input, No Output.
           static void sayHello(String name) { // yaha input ha ---> (String name).
               System.out.println("Hello " + name);
           }
       // Type-3) No Input, Output.
           static int getNumber(){// yaha pr output ha---> {return 10;}.
               return 10;
```

```
        }
    // Type-4) Input, Output.
        static int multiply(int a, int b){// yaha pr input-->(int a, int b), or output ha-->
{return (a*b)}
            return (a * b);
        }
```

Function Overloading....

```
public class FunctionOverloading {
public static void main(String[] args) {
        // Function Overloading:
          int x = sum (3, 5);
System.out.println(x); // hum normaly functin ko is tarah se use krte ha .

  // ab me agar 3 digits ko add kru to me niche function add kr skta ha
        int y = sum(3444, 432, 333);
          System.out.println(y);
          int p = sum( 23.3, 34);
          System.out.println(p);


          Out of the main
                }
        static int sum(int a, int b){
                return(a + b);
                    }
      // ab agar muj 3 values ko add krna ha to me asa krk skta hu.
static int sum(int a, int b, int c) {// kuki beshak sum same ha pr Arguments to alag
                                    ha.
                return (a - b - c);
                    }
        // or me different type of parameters bhi rakh sakta hu.
            static int sum(double r, double s){
                return (int)(r + s);
                    }
```

Channing of functions...

```
public class FunctionChanning {
public static void main(String[] args) {

  // Channing in function---> iska concept same ha Nested ki tarah .
                function1();
            System.out.println("bye");


                    }
            static void function1(){
                    function2();
            System.out.println("okay");
```

```
    }
    static void function2(){
        function3();
        System.out.println("Hello");
    }
    static void function3(){
        System.out.println("Hii");
    }

}
```

Scope of variables....

```
// One more important topic---> Scope of variable:
//it means ki koi bhi variable ha wo agar "{}" ke ander ha to vo sirf wha tk hi simit ha.
```

Recursion:

It simpaly means calling itself.
Example: printnum

```
public static void main(String[] args) {
        // Recursion in function--->

        int n = 98;
        printNum(n);

    }
        static void printNum(int n) {
            if (n==0) return;
            printNum(n-1); // Recursive call
            System.out.print(n+", ");
        }
```

Printing Fibonacci numbers by Recursion

```
public static void main(String[] args) {
        int n = 6;
        System.out.println("Fibonacci sequence upto " + n + ":");

        for(int i = 1; i<= n; i++){
            System.out.print(fibonacci(i) + ", ");
        }

    }

    static int fibonacci (int n){
        if(n<=1){
            return n;
        }
        return (fibonacci(n-1) + fibonacci (n-2));
    }
}
```

OOP (Object Oriented Programming)

OOP ka concept ha kisi particular object ke sabhi characterstics ko ek similar object me mimic krna.

Example: Student data:

String name = “Aditya”;
Int age = 23;
Int rollnumber = 10;
String college = “ GDC, Solan

Yha pr “Student” ek real world Object ha, or ye sabhi characterstics isi real world objects ke ander hi ha. It means ki sabhi variables jo alag-alag ha vo Student block ke ander hi mimic hoge OOP me.

Student	
String name	**Int rollnumber**
Int age	**String college**

Real World	Programming
Students = ye ek **ABSTRACT TERM** ha.(ism bhot **objects** aate ha→ **Tanuj, Rahul,** etc “or in sabke alg-alg **characteristics** ha.	In **ABSTRACT TERMS** ko hum **(class students)** se represent krta hu. Therefore, class is known as **blueprint** of object
	Ye class hamesha public class ke bahar hi banegi. or
	Isme jo bhi variables save hote ha unhe INSTANCE VARIABLES khete ha.
	Or isme jo bhi functions save hote ha unhe INSTANCE FUNCTIONS khte ha.

```
public class OOPintroduction {
    public static void main(String[] args) {

        Student s1 = new Student();//
        Student s2 = new Student();

        s1.name = "Rahul";
        s1.age = 18;
        s1.rollnumber = 100;
        s1.college = "G.D.C.Solan";

        s2.name = "Ansh";
        s2.age = 19;
        s2.rollnumber = 101;
        s2.college = "G.D.C.Solan";


        s1.markAttendence();
        s1.print();
        s2.markAttendence();
        s2.print();
```

```
    }
}// ha pr public class khatam ha.

class Student{//class is the blueprint of the data.
    String name; // these all are the characterstics of data / object.
    int age;   // these are known as INSTANCE VARIABLES.
    int rollnumber;
    String college;

    // these objects are known as INSTANCE METHODS.
    void markAttendence() { //these all are the behaviour of the data / object.
        System.out.println ("Attendence marked by " + name);
    }
    void print(){
        System.out.println("Name = " + name +", "+ "Age = "+ age+", "+ "Roll Number = "+ rollnumber+", " + "College
Name = "+ college);
    }


}
```

Cunstroctors in OOP

Defination: jab bhi object create ho ra hota ha to CUNSTROCTORS use value assign kr dete ha.

Rules of Cunstroctors:

1. **Same name as class.**
2. **It is also a method / function of special type→ With no return; type, and not even " void".**
3. **Automatically called during object creation.**
4. **Used to initialize an object.**
5. **It can also be overloaded.**

```
class Student { // public class ke bahar bani ha.
    String name ;
    int age;
    int rollNumber;
    String college;
// constructor method / functions below.
    Student(){ // same name as class. Taki details khali hone pr cunstrocter ki details
print ho.
        name = "Priyansh";
        age = 19;
        rollNumber = 1;
        college = "G.D.C Solan";
    }
```

Parameterized Cunstroctor:

```
  // Parameterized Constructor: it simpaly means ki hum parameters de rhe ha, uske hisab se
                                details print kro.
                        Student(String n, int a, int rN, String c){
                                        name = n;
                                         age = a;
                                    rollNumber = rN;
```

```
                                        college = c;
    }// or ab hume khali method cll nahi kr skte kuki ab hume parameters assign krne hi
                                        pdege.
       //  jab tak hum default parameter add krke cunstroctor overloading na kr le.
```

“This” Keyword

```
                class Student { // public class ke bahar bani ha.
              String name ; // niche wala “this” isko denote kr rha ha.
                                        int age;
                                    int rollNumber;
                                    String college;
           Student(String name, int age, int rollNumber, String college){
         this.name = name; // "this." is used to denote class instance variables
                                      this.age = age;
                                 this.rollNumber = rollNumber;
                                    this.college = college;
```

Simply “this” student class ke instance variables ki values ko denote krne ke liya ha tki hum same values rkh ske.

Overloading in Cunstroctor

```
class Student{
  String name;
  int age;
  int rollNumber;
  String college;

  Student(){}

  Student(String name){
      this.name = name;
  }
  Student(String name, int age){
      this.name = name;
      this.age = age;
  }
  Student(String name, int age, int rollNumber){
      this.name = name;
      this.age = age;
      this.rollNumber = rollNumber;
  }
  Student(String name, int age, int rollNumber, String college){
      this.name = name;
      this.age = age;
      this.rollNumber = rollNumber;
      this.college = college;
  }
```

Channing in Cunstroctor through ”This” keyword

```
// Channing of the above Methods----> and replace all instance methods variables by--->
// all cunstroctors are calling to the last cunstroctor:
Student(){
this("unknown", 0, 0, "unknown");
}
Student(String name){
this(name, 0, 0, "unknown");
}
Student(String name, int age){
this(name, age, 0, "unknown");
}
Student(String name, int age, int rollNumber){
this(name, age, rollNumber, "unknown");
}
Student(String name, int age, int rollNumber, String college){
this.name = name;
this.age = age;
this.rollNumber = rollNumber;
this.college = college;
}
```

Objects

<table>
<tr><th>Kon kitni storage leta ha.</th><th colspan="3">Size</th></tr>
<tr><td>Refrence Variables</td><td colspan="3">4 bytes / 8 bytes (it varies JVM to JVM) majorly 4 bytes.</td></tr>
<tr><td colspan="4">Objects kitne size lete ha. It is divided int 3 parts</td></tr>
<tr><td>1) Object Headers:</td><td>It stores Metadata.</td><td>Types of Metadata</td><td>Size</td></tr>
<tr><td></td><td></td><td>1] Mark words</td><td>8 bytes</td></tr>
<tr><td></td><td></td><td>2] Class pointer: type of refrence</td><td>4 bytes</td></tr>
<tr><td>2) Exact data</td><td>Ye vahi data ha jo us object me store ha jse (student data)</td><td colspan="2">Iska size data pr depend krta ha.</td></tr>
<tr><td>3) Padding</td><td>Java me hum koshish krte ha ki har OBJECT (8bytes ke multiple) memory le store hone me</td><td colspan="2">To jitni memory (object Header) or (Exact data) lete ha usko ye nazdiki 8bytes ke multiple tak pahuchane me jitne bytes lagte ha utne add krr deta ha.</td></tr>
</table>

Call By Value

It simply means ki jab hum kisi bhi function ko call krte ha to uske effect main variable pr ni pdta.

Java is a fully call by value language ha.

```
public class CallByValue {
public static void main(String[] args) {
// call by value:
int x = 4;
int y = 5;
System.out.println(x + ", " + y); //Result ---> 4, 5
addTen(x, y);
System.out.println(x + ", " + y);//Result ---> 4, 5 (kuki addTen  ki call values Dynamic hoti ha.)
}
static void addTen(int x, int y){
x = x + 10;
y = y + 10;

}
}
```

Deep copy

It fully copy the existing Object to a new Object.

Shallow copy

It points to the existing object or it gives reference to the existing objects.

```
Random r1 = new Random(4, 5);
Random r2 = new Random(r1);  // Deep Copy
Random r3 = r1; //Shallow copy.....no "new" keyword.
```

Static Keyword

It simply means ki jab hum kisi class ke instance variable ko memory me commonly store krte ha to uske aage "static" laga kr hum use common bna skte ha.
Isme "G D C Solan" heap memory me store ni hoge vo ab class variable ha.
Samely 'static' keyword is used by methods also .

Rules for (Methods) using (static) keyword.

- **One (static method) can only call other (static method).**
- **Static method can only access static variables.**
- **Does not have access to (this) keyword.**
- **Static ko hum (static block) me rakhte ha.**
- **Parameters can't be static.**
- **Only Nested class can be static.**

```
public class StaticKeyword {
    public static void main(String[] args) {
        Student s1 = new Student ("Mehak", 19, 01);
        Student s2 = new Student ("Priyansh", 21, 02);

        Student.college = "IIT Solan"; // isko me "static block" me assign kr skta hu.

        System.out.println(s1.name + ", " + s1.age + ", " + s1.rollNumber + ", " +
Student.college);
        System.out.println(s2.name + ", " + s2.age + ", " + s2.rollNumber + ", " +
Student.college);
```

```
    }
}
class Student{
    String name;
    int age;
    int rollNumber;
    static String college;// Static ka matlab ha ki ab ye wali string ---> class string ha.
    //static String college = "IIT Solan"; // "static block ki jagah me yaha bhi direct lik
sakta hu."
    Student(String name, int age, int rollNumber){
        this.name =name;
        this.age = age;
        this.rollNumber = rollNumber;
    }
    // Static Block
    static{
        college = "IIT Solan";
    }
}
```

Final Keyword

- **“Final”** is used to fix values.
- It makes them Constant.
- Its letters become **Upper** case and **“_”** as a separator. **Example**→final String COLLEGE_NAME;
- **Final** ke sath uski valu bhi likhni pdti ha. **(final String COLEGE_NAME = “IIT Solan”;**
- **Final can be used with variable.**
- **Final can also be used with methods.**
- **Final can also be used with class.**
- **Final can also be used with parameters.**

```
public class FinalKeyword {
    public static void main(String[] args) {
        Random r1 = new Random();
            System.out.println(r1.PI);
    }
}
class Random{
    final double PI = 3.14;//use of final to fix the value.

    // we can use "final" and "Static" together.
    static final String COLLEGE_NAME = "IIT Solan";// it makes it constant as well as
accessable to all.
}
```

PILLARS OF OOP’S

1) Encapsulation

- **Both data and Behaviour should be together (encapsulated) Within an object.**
- **We should not provide unrestricted access of data. Means should provide restricted access of data.**

Access modifiers

Access Modifiers controls who have access of variable, method, cunstroctor, class?

4 Types of Access Modifiers

- **(public)** Access Modifier—> Anyone can access, (most unrestricted).
- **(protected)** Access Modifier—> Only same package access + Inherited class (child class) access.
- **(default)** Access Modifier—>Anyone can access belonging to same package. Ye defaultly lag jata ha.
- **(private)** Access Modifier—> no one can access out side of the (class).

```
public class Encapsulation {
    public static void main(String[] args) {
      BankAccount ba = new BankAccount ();
        ba.deposit(500);
        ba.withdraw(200);
        System.out.println(ba.getBalance());

    }
}
class BankAccount{
    private double balance;// "private" only can be accessed inside the class(BankAccount);


    public void deposit (int amount){// 'public' anyone can access.
        balance += amount;
    }
    public void withdraw (int amount){
        balance -= amount;
    }
    // geters / setters
    public double getBalance (){
        return balance;
    }
}
class Student{
    private String name;// private Access modifier.
    private int age;
    private int rollNumber;
    private String college;

    Student(String name, int age, int rollNumber, String college){
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
    }
    // getters / setters---> inka kaam proper validations lagana ha taki koi bhi unnecessary values
na daale.
    public String getCollege(){  // public access modifier.
        return name;
    }
    public void setCollege(String college){
        // validations ---> college should be real.
        this.college = college;
    }
}
```

“Package”

Note: **“Package”** directly means ki kisi bhi “.java” vali file ko kisi particular folder me dalna.

- Kisi bhi new file me agar kisi package ki file ki class ko dalna ha to uske liye muj (**import packageName.className**) dalna hoga.
- Or agar muj kisi package ki sabhi class ko ek sath add krna ha to top me (**import packageName.*)** dalna hoga.
- And these above packages are known as **USER DEFINED PACKAGES.**
- Whereas java has **In-built packages** also in their LIBRARIES, for **example**→
  java.util
  java.lang
  java.io

---

2) Inheritance

- It shows the relations between two classes of single package’s file.
- It shows **“is-a”** relationship, Example: Vechile→ cars, bike, etc.
- Showing that car **“is-a”** vehicle.
- Programatically it defined by→ **(extends)** keyword.
- It gives relief of **code reusibility.**
- It helps to support polymorphism.

Types of inheritance are shown below.

1) **Simple inheritance**→ One Parent class→One child class.

```
class Student { // parent class.
    String name;
    int age;

    void markAttendence(){
        System.out.println("Attendence Marked");
    }
}
class EngineeringStudent extends Student{ // child class.
    //no need to add 'markAttendence()' because it is a child class of 'Student'.
    void attendLab(){
        System.out.println("Lab attended");
    }
}
```

2) **Multi-level inheritance**→ (parent→ child→child’s child.).

```
class Student { // parent class.
    String name;
    int age;

    void markAttendence(){
        System.out.println("Attendence Marked");
    }
}
class EngineeringStudent extends Student{ // child class.
    void attendLab(){
        System.out.println("Lab attended");
    }
}
class CSEengineeringStudent extends EngineeringStudent{  // child's child
}
```

3) **Hierarchical inheritance**→ One parent class→ child class 01, child class 02.

```
class Student { // parent class.
    String name;
    int age;

    void markAttendence(){
        System.out.println("Attendence Marked");
    }
}
class EngineeringStudent extends Student{ // child class.
    void attendLab(){
        System.out.println("Lab attended");
    }
}
class MedicalStudent extends Student{  // another child class
}
```

4) **Multiple inheritance**→ Parent-1 class, Parent-2 class—> common child class.

- Not supported in java, due to diamond problem.

Super keyword

- It is used to access parent class's **variable**.

```
class Student { // parent class.
    String name;
    int age;
    int rollNumber;
    int x;
}
class EngineeringStudent extends Student{ // child class.
   String college;
    int x;
   void Print() {
    System.out.println(super.name + ", " + super.age + ", " + super.rollNumber + ", " + super.x);
    //calling the variables of parent class.
   }
}
```

-
- It can call to the **methods** of parent class.

```
class Student { // parent class.
    String name;
    int age;
    int rollNumber;


    void Print() {
    System.out.println(name + ", " + age + ", " + rollNumber );}
}
class EngineeringStudent extends Student{ // child class.
   String college;

   void Print() {
    super.Print();
    System.out.println(college);
   }
}
```

- Can call parent class **constructor**, by→ **super.();** // almost similar to methods calling.

---

3) Abstraction

Defination: The process of focusing on what something does, while ignoring how it does that, just by only representing whatever is necessary.

Types of Abstraction

- **Low level Abstraction**→ Hiding implementation details. **Example**→ jab bhi hum koi 'method' call krta ha to hum uske under ke deep implementation ko nahi dekhte ha, kii us method ke ander kya ha.

- **High level Abstraction**→ Seprate what from how. By using (**Abstract class** & **Interfaces**). It simpally means→hum ek class ki 2-child class bna dete ha. Jisko hum bolte ha **Hierarchical inheritance**

- **Abstract class** is a **keyword** jo hume parent class ke method ko sirf **declare** krne me help krta ha, taki define nahi krne pde Parent class me.
  { **accelerate();** } hum ase nahi likh skte, pr hum { **abstract accelerate();** } ase likh skte ha.

- Or jis class ka agar koi bhi **method** abstract ha to wo puri **class** ko abstract banana pdega. { **abstract class car**}. It shows the **families** of similar objects. Isme ko compulsory **contract** nahi ha.

```
// HIGH LEVEL ABSTRACTION BY "abstract" keyword.

        car c = new fuelCar();// yha pr me direct 'car' class ka object nahi bna skta, kuki car class "abstract" ha.
            c.start(); // calling to Parent class "car"
            c.accelerate();// calling to child class "fuelCar".
            c.brake(); // calling to child class "fuelCar".
    }
}
abstract class car {// kuki ander "abstract methods" ha to class ko bhi "abstract" banana zaruri ha.
    void start(){
        System.out.println("car started");
    }
    abstract void accelerate(); // here "abstract" keyword is used to only declare the method for child classes.
    abstract void brake();
}
class fuelCar extends car{ // kuki Parent class me "accelerate" or "brake" methods sirf 'declare' ha
//  to child class me hume 'define' bhi krna pdega.
    @Override // ye compulsory nhhi ha bs good practice ha, or ue batata ha ki hum is method ko alag se impliment kr
re ha.
    void accelerate(){
        System.out.println("accelerate fuel car");
    }
    @Override
    void brake(){
        System.out.println("fuel car is breking");
    }
}
class electricCar extends car{
     @Override
    void accelerate(){
        System.out.println("accelerate electric Car");
    }
     @Override
    void brake(){
        System.out.println("Electric car is braking");
    }
}
```

Questions about Abstract classes

- Can abstract classes have constructor —> yes.
- Can abstract classes be final —-> no.
- Can abstract classes have static variable/method —>yes.
- Can abstract classes have private method.---> yes but ye non-abstract hone chahiye.
- Can abstract classes have final method—> yes but ye bhi non-abstract hone chahiye.
- Can abstract classes have no abstract method —->yes.

Interface class of abstraction

- **Interface** is a necessary **contract**.
- Isme **abstract** keyword lagane ki zarurat nahi hoti **declare** krne k liye.
- It gives **responsibilities, roles, capabilities.**
  Therefore java ki jo apni **interfaces classes** hoti ha unke naam me **" able"** laga hota ha→ **Example:** comparable, serializable. It shows ki ye kya kr skta ha ?
- We can only **declare** methods in parent class.
- Child class use **"implements**" keyword to join child class with parent class.
- Child class me parent class ke sabhi methods ko define krna zaruri ha kuki vo unke roles ha.
- It is a pure **"what"** language. It means ki hum class me sirf **declare** krte ha uski under detail me nahi jante ha.
- It is good practice to add **"I"** before any interface class. Taki normal class or interface class me farak pta chl sake.

```
interface Car { // interface keyword is  added
    void start(); // cant define methods here.
    void accelerate();
    void brake();
}
class fuelCar implements Car { // implements keyword is addid to join with Parent class.
    public void start(){
        System.out.println("Fuel Car started");// necessary to define all methods of parent
class.
    }
    public void accelerate(){
        System.out.println("Fuel Car is accelerating");
    }
    public void brake(){
        System.out.println("Fuel Car is stopping");
    }
}
class electricCar implements Car {
    public void start(){
        System.out.println("electric car is started");
    }
    public void accelerate(){
        System.out.println("electric car is accelerating");
    }
    public void brake(){
        System.out.println("electric car is stopping");
    }
}
```

Polymorphism

- Poly → many, morphism→ forms.
- It means same object behaves differently on same command if one parameter is changed.
- It is possible by “method overloading”. And this is knon as ‘compile-time polymorphism’.
- Jab Parent class se 3 child class ko same method mille (method @overriding) to wo run-time polymorphism hota ha.
- Fields / variables can’t be polymorphic, they don’t participate in overriding.

Wrapper Classes

- Har **primitive** datatype ki ek **wrapper class** hoti ha, or vo class **non-primitive** hoti ha.
  Example: int = integer class, short = short class, float= float class.

Autoboxing / Unboxing

Autoboxing:

- It automatically change **“primitive datatype”** to their respective WRAPPER CLASSES **“non_primitive datatype”**.
  Example: int x = 10; —>Integer y = x;
- Internally kucha asa hota ha→ **Integer y = new Integer (x);** // old java internally perform this.
  Integer y = Integer.value of (x); // new java internally perform this.

Unboxing:

- It automatically change **“Wrapper classes** to their respective **primitive datatypes.**
- **Example:** Integer x = 10; —> int y= x;
- Internally kuch assa hota ha.---> **int y = x.intValue();**

- **Autoboxing & unboxing is done with Assignments.**
- **Autoboxing & unboxing is done with Methods calls.**
- **Autoboxing & unboxing is done with Arithmatic operations.**
- **Primitive can’t store “null”.So we can’t store wrapped classes default null in primitive datatype.**

Que) Why Integer x= 200; / Integer y = 200; ---> System.out.println(x==y); ---> false.

Ans) “==”primitve datatype ke variables ke liye **values** ko compare krta ha, pr uper wale case me **“==”** Wrapped class ki values ke refrence ko compare kr ra ha jo stack memory me ha. Or refrence ke ander value ka address store hota ha. To “==” x,y ke value ke address ko compare kr rha ha, na ki value ko.

```
public class Autoboxing {
    public static void main(String[] args) {
        int x = 100;
        int y = 100;
            System.out.println(x==y);// Ans--> true.

        Integer a = 200;
        Integer b = 200;
            System.out.println(a==b);// Ans--> false.(hum values ko nahi balki x,y refrence
ke address ko compare kr rhe ha).
            // Agar muj values ko compare krna ha to muj asa krna hoga-->
            System.out.println(a.intValue()== b.intValue());// true.
            // or me ya bhi krr skta hu.
            System.out.println(a.equals(b)); // true.
        Integer p = 100;
```

```
        Integer q = 100;
            System.out.println(p==q);// Ans----> true. (it use cashing)
            // matlab (int -128-----127) tk ki values ke liye fix refrence hota ha,
            //  to jab hum call krte ha to direct unki values call hoti ha.
    }
}
```

POJO CLASS “plain old java object”

- Isme koi hardcore business logic nahi hona chihiye.
- Isme sirf getter/setter, constructoer, variables(field).

Nested Class

Nested classes are of 4 types:

1. **Static Nested Class.**

```
public class NestedClass {
    public static void main(String[] args) {
        Outer outer = new Outer(); // to access outer class
        Outer.Inner inner = new Outer.Inner();// to access the nested innerr class.
        inner.fun();// me inner nested class ki function ko call kr dega.
    }
}
class Outer{
    static class Inner{ //ye nested class static ha to isko access krne k lia hum outer class ke object ki zarurat
nahi ha.
        void fun(){
            System.out.println("Hello");
        }
    }
}
```

- Static inner class can’t access the non-static members of the outer class directly.
- To get access we have to take parameters in methods or constructors of inner class.
- Inner class behaves same as normal class.
- Inner class ka nam uski outer class ke sath hi juda hoga, example→Outer.Inner.
- It can act as helper class for outer class.
- It is used in builder design pattern.
- If you want to have static methods inside a nested class.
- Request / Response DTO.

---

2. **Inner Nested Class:** The class which is not static-nested class.

```
public class InnerNestedClass {
    public static void main(String[] args) {
        Outer outer = new Outer(); // outer class ka object simple hi bnega.
        Outer.Inner inner = outer.new Inner();// yaha pr nested class k lia
        //                                    "new" se phle hume "outer" referance likhna zaruri ha.
        inner.fun();
    }
}
class Outer{
    int x =10;
    class Inner{ // is class me static nahi ha to ye outer class ke object pr depend krti ha.
        int x = 20;
        void fun(){
            System.out.println(x);
            System.out.println(Outer.this.x); //to call outer class variable.
```

```
        }
    }
}
```

- It can access Static and non-Static members of the outer class.

---

3. **Local Nested Class:** koi bhi asi class jo hum kisi code block ke ander bna dege, Example→ if, for, while, switch, function, method, constructor, static block, etc.

```
public class LocalNestedClass {
    public static void main(String[] args) {
        Outer outer = new Outer();
        outer.greet();
    }
}
class Outer {
    private int x = 4;
    void greet(){// is method ke ander nested local class ha.
        int y = 5;// me is local variable ko is local class me use to kr skta hu pr,
        //y++;// me is "y" variable ko modify nahi kr skta. {EFFECTIVE FINAL VARIABLE}rule se.
        class Local{ // is class ko local nested class khte ha.
            void sayHello(){
                System.out.println(y);
            }
        }
        Local local = new Local(); // local class ka object hmesha resoective orgainization ke ander hi rhega.
        local.sayHello();
    }
}
```

Bhot kum use case ha local nested class ka.

- It uses **effective final variable** rule:

---

4. **Anonymous Nested class:** A class without Name is called Anonymous class.

```
public class AnonymusNestedClass {
    public static void main(String[] args) {
        Person p1 = new Person(){ // this is a anonymous class with no name.
            @Override
            void introduce(){
                greet(); // muj greet yhi call krna hoga .
                System.out.println("Hii i am Priyansh");
            }
            void greet(){
                System.out.println("Hello"); // me bahar greet call nhi kr skta.
            }
        };
        p1.introduce();
       ;
    }
}
class Person {
    void introduce(){
        System.out.println("HI I am a guest");
    }
}
```

- This class automatically extends the class.
- This class can’t call constructor, kuki usme object name nhi ha.

Benefits of Nested classes:

1. Logical grouping.
2. Get better access of Outer class, example→private keyword, therefore it does not requires any getters/setters of Outer class.

Standard Input / Output

Types of Input / Output

1. **Console I/O**
2. **File I/O**
3. **Network I/O**
4. **Memory I/O**

Console I/O: Jo bhi hamari screen pr print hota ha use console khte ha.

System.out.println("Console Output");

System.err.println("Console Error"); to print error statements.(good Practice).

```
public class SystemINxOP {
    public static void main(String[] args) {
        // System.out.println defination:
        System.out.println("Hello");
    }
}
/*
             java.lang package:---> ye defaultly import ho jata ha, main class mein
class System{
    public static final PrintStream out;
}


            java.io package:
class PrintStream{
    public void println(String s){
        code for printing
    }
}
*/
```

- **println**→ new line.
- **Print**→ without new line.
- **Printf**→ print formatted output.

---

system.in- to read data from keyboard.

```
             java.lang package:---> ye defaultly import ho jata ha, main class mein
class System{
    public static final InputStream in;
}


            java.io package:
class InputStream{
    public void read(String s){
        Reads input
    }
}
```

Keyboard input before Scanner class.

Phle hum java ki jagah ye use krte the→
keyboard—>OS Buffer—>Input Stream→ Input stream reader→Java Buffer →readline methos→Program
(buffer is type of arrays that stores characters.)

1. **Take Input from keyboard "Priyansh"**
2. **OS Buffer (23,243,354,5,334,34,65,6,)takes in integer due to ASCII, UNICODE.**
3. **(inputStream object) receives bytes from OS.**
4. **InputStreamReader: converts stream of bytes into stream of char.(P,r,i,y,a,n,s,h)**
5. **Buffered Reader: will read all character inputs, and stores in java buffer, in string format.**
6. **Program: jab bhi java ko zarurat hogi to wo Buffered reader se le-lenga, by readline();**

Why Scanner class

- **Simplify (input)**
- **Read without typecasting all → int, double, boolean, string.**

Scanner

- **Scanner class comes in (java.util) Package, it means it is a utility class.**
- **Scanner ka object apne constructor ke parameter me InputStream(System.in) expect krta ha.**
- **constructor ka parameter decide krte ha ki Scanner ky input lega.**
- **Example Input from keyboard→System.in, Input from file→new file(file name), etc.**
- **Scanner internally works on tokenization basis.**
- **Scanner class is slower than Buffer reader .**
- **For more optimized code we use buffer reader, example→ competitive programming.**

Methods of SCANNER:

- **next→ It only reads a word.**
- **nextline→ It reads whole line, jabtak hum enter press nhi kr dete.**
- **nextInt→ it reads integer values.**
- **nextDouble→it reads double values.**
- **Etc, all type of datatypes.**

```
import java.util.Scanner;
public class ScannerClass {
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);// new scanner object with parameter of
Keyboard input.
        int age = sc.nextInt(); // methods of scanner.
        double num = sc.nextDouble();
        String name = sc.next();
        String college = sc.nextLine();
    }
}
```

Immutable Classes

Rules for Immutable classes:

- **Mark my class as final.**
- **Mark my instance variable as final.**
- **Mark my instance variable as private.**
- **No setters.**
- **Defensive copy in getters/constructors→non-primitve ko alag se create kro or uska refrence de do.**

- **Immutable objects are important in threads.**

```java
public class ImmutableClasses2 {
    public static void main(String[] args) {
        College college = new College("GDC Solan", "Himachal Pradesh");
        Student s1 = new Student(28, "Priyansh Sharma", college);
           System.out.println(s1.getCollege().name); // GDC Solan
        s1.getCollege().name = "IIT Solan"; // if we change the name of college.
           System.out.println(s1.getCollege().name);// IIT Solan.
    }

}
//  purelly Immutable class.
// defensive copy of non-primitive (college).
final class Student{
    private final int age;
    private final String name;
    private final College college;

    Student(int age, String name, College college){
        this.age = age;
        this.name = name;
        this.college = new College(college.name, college.address);// defensive copy.
    }
    // getters
    public int getAge(){
        return this.age;
    }
    public String getName(){
        return this.name;
    }
    public College getCollege(){
        return new College(this.college.name, this.college.address);//defensive copy/deep copy.
    }
}
class College{ // normal/mutable class.
    String name;
    String address;

    College(String name, String address){
        this.name = name;
        this.address = address;
    }
}
```

Object Classes

- **This class is in java.lang package.**
- **It is parent class of all classes in java, it means every class in java defaultly inherits from object class.**
- **Object class me hum common behaviours rkhte ha jo hum har class ke liye commonly zaruri ha.**
- **Object class ka reference kisi bhi class ko or unke object ko point kr skta ha, because of polymorphism.**

Core methods of object class:

1. **toString():** converts anything to String.

```java
public class toStringMethod {
```

```
    public static void main(String[] args) {
       //Practicing toString method--->
       Student s1 = new Student();
       s1.name = "Priyansh";
       s1.age = 24;
       System.out.println(s1.toString());// agr me sirf println(s1) bhi likhu tb bhi ye call ho jayga.
       System.out.println(s1); // both means same. result will be same.
    }
}
class Student{ // every class defaultly extends Object class.
    String name;
    int age;
        @Override
        public String toString(){
            return (name + ", " + age);
        }
}
```

2. **equals():** compare 2 objects & returns true/false (boolean value).

```
public class equalsMethod {
    public static void main(String[] args) {

        // practicing equals Method shows boolean value.
         Student s1 = new Student();
            s1.name = "Priyansh Sharma";
            s1.age = 23;
         Student s2 = new Student();
            s2.name = "Priyansh Sharma";
            s2.age = 23;
         System.out.println(s1.equals(s2));// internally--> Object obj = new Strudent();
    }
}
class Student{ // every class defaultly extends Object class.
    String name;
    int age;
        @Override
        public boolean equals(Object obj){
            if(this == obj) {
                return true;
            }
            if (obj == null){
                return false;
            }
            //check if both classes are of type Student.
            if(obj.getClass() != this.getClass()){
                return false;
            }
            Student s = (Student) obj;// typecast obj to student, therefore s2 is now
student
            return (this.name == s.name && this.age == s.age);
        }
}
```

3. **hashCode():** Return an integer in hexadecimal format of an Object.

Rules of hashCode():

- **If 2 objects are equal than their hasCode must be equal.**
- **But the reverse might not be true.** It means if hashCode of 2 different objects are equal it dosen't means the objects are equal.

```
import java.util.Objects;
public class hashCodeMethod {
    public static void main(String[] args) {
        // practicing hashCode Method shows boolean value.
         Student s1 = new Student();
            s1.name = "Ram Chauhan";
            s1.age = 25;
         Student s2 = new Student();
            s2.name = "Ram Chauhan";
            s2.age = 25;
         System.out.println(s1.equals(s2));// internally--> Object obj = new Strudent();
         System.out.println(s1.hashCode() == s2.hashCode());// therefore hashCode are
equal--> True
         System.out.println(s1.hashCode());// to check s1 hashCode.
         System.out.println(s2.hashCode());// to check s2 hashCode.
    }
}
class Student{ // every class defaultly extends Object class.
    String name;
    int age;
        @Override
        public boolean equals(Object obj){
            if(this == obj) {
                return true;
            }
            if (obj == null){
                return false;
            }
            //check if both classes are of type Student.
            if(obj.getClass() != this.getClass()){
                return false;
            }
            Student s = (Student) obj;// typecast obj to student, therefore s2 is now
student's object.
            return (this.name == s.name && this.age == s.age);
        }
        @Override
        public int hashCode(){
            return Objects.hash(name, age);  // here "Objects" is another class than Object
class
        }                                   //   "Objects" class java.util package me ha.
        //                                    to isko uper import krna hoga.
}
```

4. **getClass():** Return runtime class of an object.

- **Instance Operator** is also covered below.

```
public class getClass {
    public static void main(String[] args) {
        //practicing getClass Method
        Animal a = new Animal();
        Animal d = new Dog();
            System.out.println(a.getClass().getName());// Ans-->Animal. Runtime class animal
ha.
            System.out.println(d.getClass().getName());// Ans-->Dog. Runtime class Dog ha.
        //Practicing "instance Operator"--> if an object is "instance" of class or any oof
its sub-class.
            System.out.println(a instanceof Object);//true
            System.out.println(a instanceof Animal);//True
            System.out.println(a instanceof Dog);   //False
            System.out.println(d instanceof Animal);//True
    }
}
class Animal{
}
class Dog extends Animal{
}
```

Cloning method:

1. **clone():** It helps to create copy of an object.

- This method is protected type in Object class.
- To @Override this clone method in any class, that class should be “clonable interface”.
- Not every object should be Clonable,therefore it uses “clonable interface”.
- Clonable interface is empty, these type of interfaces are known as **MARKER INTERFACE.**

```
public class cloneMethod {
    public static void main(String[] args) throws CloneNotSupportedException{
        Student s1 = new Student();
            s1.name = "Priyansh";
            s1.age = 23;
        Student s2 = (Student) s1.clone(); //Typecaste to Student class from object class.
            System.out.println(s2.name);// Priyansh
            System.out.println(s2.age);// 23
    }
}
class Student extends Object implements Cloneable { // to override clone, class should be
Clonable "interface"
    String name;
    int age;

        @Override
        protected Object clone() throws CloneNotSupportedException{//it is protected.
            return super.clone();// 'Super' to access parent class variable.
        }
}
```

Garbage collection method:

1. **finalize(); It is not used now. It is deprecated now.**

Threads methods:

1. **wait();**
2. **notify();**
3. **notifyAll();**

ENUM- Enumerations (Enumerated type)

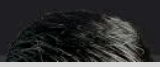
- enum Direction{ // this is an enum class.
  North, // these are the constants.
  South,
  East,
  West, }

- **Enum** is a pre-defined set of **constants**.
- Enum is nothing but a special type of class.
- Internally => **enum Direction→ class Direction.**
- Every enum class **@Override** the parent Enum class which is in java.lang package, internally.
- Example: enum Direction extends Enum <Directions> { }
- Every **constant** in enum is the object of the **enum name**.
- Each **Constants** in enum are internally by-default **static & final**.
- Default Type of each **constants** are the **enum Direction.**
- Therefore, internally it is like this.--> (**north constant**) of direction class is internally stored like this below, and same for South, East, West.
  public static final Direction North = new Direction(); // ye object type ha to yhi inke object bn gye ha.
- Internally 'enum Direction' ka apna private constructor hota ha taki koi bhar se isaka object na bna ske.

```
    public static void main(String[] args) {
        Direction d1 = Direction.NORTH;
            System.out.println(d1.getDegree());
        Direction d2 = Direction.SOUTH;
            System.out.println(d2.getDegree());
    }
}
enum Direction{ // this "Direction" is also a class.
    NORTH(0),   // these all constants are the objects of the "Direction Class".
    SOUTH(180),
    EAST(90),
    WEST(270);

    private int degree; // this should be 'private' and call through getters.

    Direction(int degree){ // constructor of enum is always private, to hum sirf isko enum ke ander
hi edit kr skte ha.
        this.degree = degree;
    }
    public int getDegree(){ // getter to call the 'int degree'.
        return this.degree;
    }
}
```

```java
enum Direction {

    NORTH(0),
    EAST(90),
    SOUTH(180),
    WEST(270);

    private int degrees;

    Direction(int degrees) {
        this.degrees = degrees;
    }

    public int getDegrees() {
        return degrees;
    }
}
```

```java
final class Direction extends Enum<Direction> {

    public static final Direction NORTH = new Direction(0);
    public static final Direction EAST = new Direction(90);
    public static final Direction SOUTH = new Direction(180);
    public static final Direction WEST = new Direction(270);

    private int degrees;

    private Direction(int degrees) {
        this.degrees = degrees;
    }

    public int getDegrees() {
        return degrees;
    }
}
```

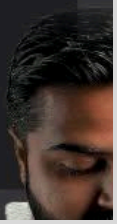


Enum Direction Moves

```java
public class EnumDirectionMove {
    public static void main(String[] args) {
        Direction d = Direction.NORTH;
        d.move();
    }
}
enum Direction {
    NORTH{ // Anonomyus class--> with no name.
        @Override
        public void move(){
            System.out.println("Move up (Y+1)");
        }
    },

    SOUTH{
        @Override
        public void move(){
            System.out.println("Move down (Y-1)");
        }
    },
    EAST{
        @Override
        public void move(){
            System.out.println("Move right (x+1)");
        }
    },
    WEST{
        @Override
        public void move(){
            System.out.println("Move left (x-1)");
        }
    };

    public  abstract void move(); //Abstract Method, or isme hume class ko abstract krne ki zarurat
nahi ha.
}
```

```
enum Direction {
    NORTH {
        @Override
        public void move() {
            System.out.println("Moving Up");
        }
    },
    SOUTH {
        @Override
        public void move() {
            System.out.println("Moving Down");
        }
    },
    EAST {
        @Override
        public void move() {
            System.out.println("Moving Right");
        }
    },
    WEST {
        @Override
        public void move() {
            System.out.println("Moving Left");
        }
    };
    public abstract void move();
}
```

```
final class Direction extends Enum<Direction> {
    public static final Direction NORTH = new Direction() {
        @Override
        public void move() {
        }
    };
    public static final Direction EAST = new Direction() {
        @Override
        public void move() {
        }
    };
    public static final Direction SOUTH = new Direction(){
        @Override
        public void move() {
        }
    };
    public static final Direction WEST = new Direction(){
        @Override
        public void move() {
        }
    };
    public abstract void move();
}
```



class Enum→ default methods in “class Enum”.

- **values():** myth that this comes from class Enum but this method is compile generated.
- **calueof(String);** this is also a myth that it comes from class Enum and this is also a compile generated method.
- **name():** Yes, this is a method that comes from enum class, it is used to get name of the enum Direction→ constants values→ EAST, WEST, NORTH, SOUTH., and this method can’t be override, because it is final.
- **ordinal():** Yes, this is also a method that comes from enum class.

```
public class MethodsOfEnumClass {
    public static void main(String[] args) {

        //IF i call values();--> this is out of enum class, it is compiler generated method.
        // this is used taki hum "enum Direction" ki sari values laa sake.
        Direction[] directions = Direction.values();
            for(Direction d : directions){
                System.out.println(d.name());
            }
        //valueof();--> this is also out of enum class, it is also compile generated.
        //ye sirf string ko constant me convert krta ha, or ye case sensative hota ha.
        Direction d1 = Direction.EAST;
        Direction d2 = Direction.valueOf("EAST"); // CASE SENSATIVE.
            System.out.println(d2.name());
        // name();--> it is a method from enum class, this is final can't be @override.
            System.out.println(d1.name());
            System.out.println(d2.toString()); // both are same but this can be @override.
        // ordinal();--> this gives the value of constants.
            System.out.println(d1.ordinal());// ans--> 2 because "EAST" is in 2nd place.
    }
}
enum Direction{
    NORTH,
    SOUTH,
    EAST,
    WEST;
}
```

Interface - Deep drive

- **Interfaces are the blue print of behaviour.**
- **Any class that implements INTERFACE should necessarily define the each behaviors.**
- **Therefore INTERFACE is also known as CONTRACT.**
- **Every method in interface is by-default public.**

Dynamic Polymorphism ka bhi ache se use hota ha, its code is given below.

```
public class DynamicPolymorphism {
    public static void main(String[] args) {
        Payment p = new DebitCard();// dynamic action yha hoga, jo bhi class dalegi uska hi method call hoga.
        p.pay();
    }
}
// Dynamic polymorphism or Dynamic Dispatch.// it means ki ye run time pr decide krta ha.
interface Payment{
    void pay();
}
class CreditCard implements Payment{
    @Override
    public void pay(){
        System.out.println("Paying via Credit Card");
    }
}
class DebitCard implements Payment{
    @Override
    public void pay(){
        System.out.println("Paying via Debit Card");
    }
}
```

Variables inside interface

- **Vo variables defaultly " public static final" hote ha.**

```
public class VariablesInInterfaces {
    public static void main(String[] args) {
        MathConstant r1 = new Random();
            r1.fun();
            System.out.println(MathConstant.VALUE);
    }
}
// Variables inside Interfaces-->
interface MathConstant {
    double PI_VALUE = 3.14;// ye ek constant ha to interface me apne aap "public static final" internally lg jata ha.
    int VALUE = 10;
        void fun();
}
class Random implements MathConstant{
    public void fun(){
        System.out.println(PI_VALUE +", "+ VALUE);
    }
}
```

Multiple inheritance interface

```
//  Multiple inheritance can be possible through interfaces--> Parent-A and Parent-B both are extended by child.
interface A {
    void fun();
}
interface B {
    void fun2();
}
```

```
class C implements A, B{
    @Override
        public void fun(){
        }
    @Override
        public void fun2(){
        }
}
```

Interface Inheritance

```
 // interface inheritance.
interface Animal{
    void eat();
}
interface Dog extends Animal{//when interface inheritance another interface the keyword used is "extends".
    void bark();
}
class StreetDog implements Dog{
    @Override
        public void eat(){
            System.out.println("eating");
        }
    @Override
        public void bark(){
            System.out.println("barking");
        }
}
```

“Default Method”, “static Method”, “private access modifier.

- **Each and every method in interface are by default “ public abstract void drive(); ”**
- **But after java 8, we can also declare the method in interface.**
- **“Default” keyword is added to declare the method in interface.**

```
interface Vehicle{
    //"Default Method".
        default void drive(){// defalult keyword is used to define the method.
            System.out.println("Vehicle is driving");
            accelerate(); // private will call through here.
        }
    //"Static Method".
        static void brake(){
            System.out.println("vehicle is applying brake");
        }
    //"private Method".
        private void accelerate(){// because this is private, so interface k bhar se call ni
kr skte.
            System.out.println("Vehicle is Accelerating");
        }
}
class Car implements Vehicle{
    // i can also @override the drive(); method, if i want.
}
```

Diamond problem fix by interface: MULTIPLE INHERITANCE

```java
public class DiamondProblemInterface {
    public static void main(String[] args) {
        D d = new D();
        d.fun();
    }
}
interface A {
    void fun();
}
interface B extends A {
    default void fun(){
        System.out.println("B interface.");
    }
}
interface C extends A {
    default void fun(){
        System.out.println("C interface.");
    }
}
class D implements B, C {
    @Override // kuki B, C dono interfaces ke method defined ha to yha @override krna hi pdega.
        public void fun(){
            System.out.println("class D.");
            // now if i want to call fun method of B, C then-->
            B.super.fun();
            C.super.fun();
        }
}
```

Resolution priority rule:

```java
public class ResolutionPriorityRule {
    public static void main(String[] args) {
        C c = new C();
        c.fun();
    }
}
/*
java resolution priority rule-->agar class c exteds class B and also implements interface A ,
and both "class B & interface A " has same method,
so, priority will be given to the class method always by default.
*/
interface A {
    default void fun(){
        System.out.println("Inside interface A");
    }
}
class B {
    public void fun(){
        System.out.println("inside class B");
    }
}
class C extends B implements A {
//agar me chahta hu ki interface ka method call ho to-->
    @Override
    public void fun(){
        A.super.fun();
    }
}
```

Interfaces	Abstract Classes
No variables are allowed, because of public static and final.	**Variables are allowed.**
No Constructors are allowed.	**Constructors are allowed.**
Multiple inheritance is possible	**Multiple inheritance is not possible.**
Methods in inheritance are generally public.	**Methods can be of any access modifier.**

Functional interfaces: These interfaces unlocks important concept of "functional programming", using **LAMDA** expressions.

- Ye wo interfaces hote ha jinme sirf ek hi functional method ho, or bs wo declare ho.
- Example: **interface A {**
  Void fun(); }
- **Some examples are**→ comparable interface, predicate interface, etc.

Marker interfaces: these interfaces are totally empty. With no methods inside it.

- Example: **interface A { }.**

Types of Marker interfaces→

1. **Interface Clonable.**
2. **interface Serializable.**
3. **Interface RandomAccess.**

Note:

- **Interface Animal →internaly when compile java code then it will convert to "class Animal" or isko java is tarah se baki class se alag krega ki usme ACC_Interface ka tag lga dega, jo ye btaega ki ye koi class nahi blki interface ha.**

Strings - DeepDrive

Character: Any symbol than is readable by humans, "a", "A", "1", . , kisi bhi language ka alphabet.
String: It is sequence of characters, it means it is the "chr []"--> character array.

- String is a non-primitive datatype.
- So string is a class, that comes under java.lang package.
- java.lang package is auto imported.

Properties of String class:

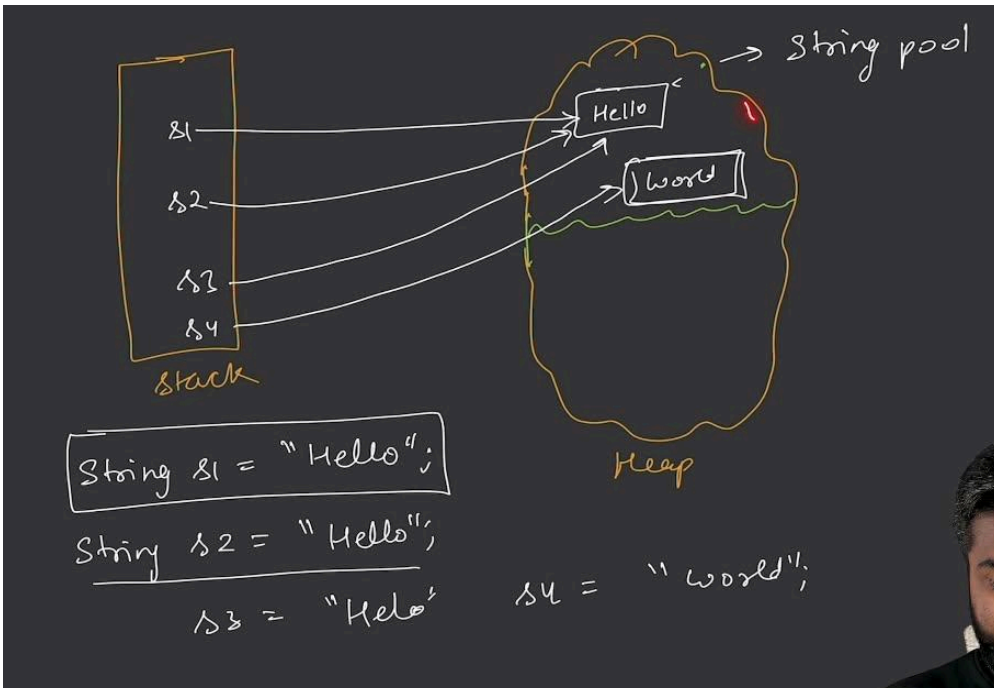
- String class is a **immutable** class, it means we can't change its any behaviour or data.
- Reason behind the immutable of class String is to be secure.

```
public class StringImmutable {
    public static void main(String[] args) {
        // String immutabilty check.
        String s1 = "Hello";
        s1.concat(" world");// too concat->string ke piche string lagana ke liye ha prr...
            System.out.println(s1); //Result-->Hello-->ye concat string me possible nahi ha,
            // kuki string immutablle class ha ,
            // orr "Hello world" alag object bn gya ha jo abhi khi point nahi ha.
    }
}
```

2 ways to declare String class objects→

1. **Using new operator:** iske literal ki copy bhi string pool me store hoti ha, bina kisi refrence ke bhi.
2. **Using literal**→ “ Priyansh” it unlocks a new feature of java “STRING POOL”.

String Pool: ye heap memory me alog se ek jagah ha jisme literl objects store hote ha, or agar koi reference ka same literal ha to wo same hi literal ko point krege na ki new object bnayega.

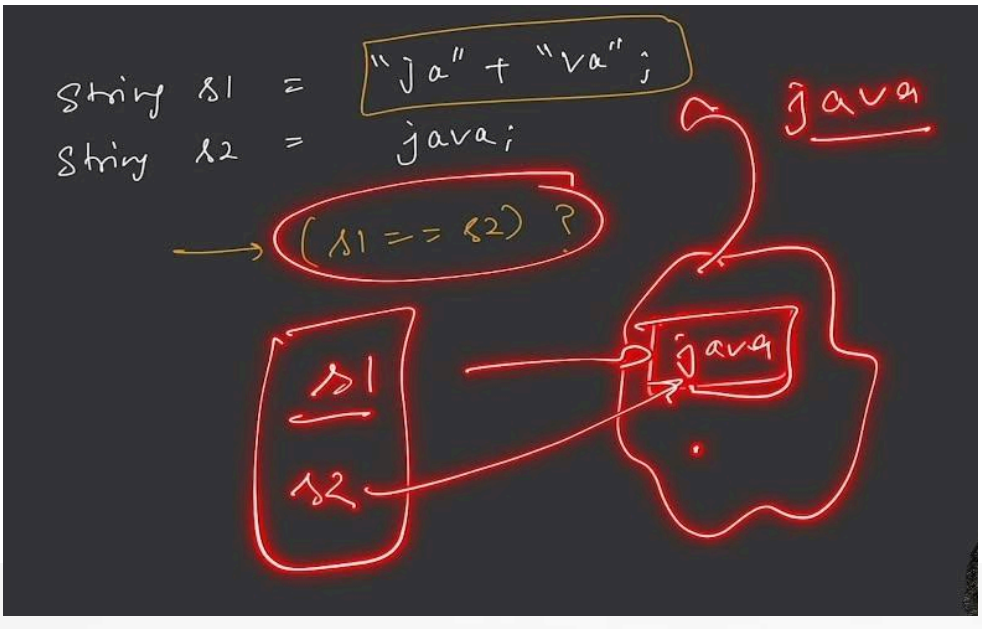


- It means ki hum strings ko reuse krte ha.
- This is part of heap memory in modern java but,
- It was not part of heap memory before,
- There was a special memory (PremGen) where String pools were stored.

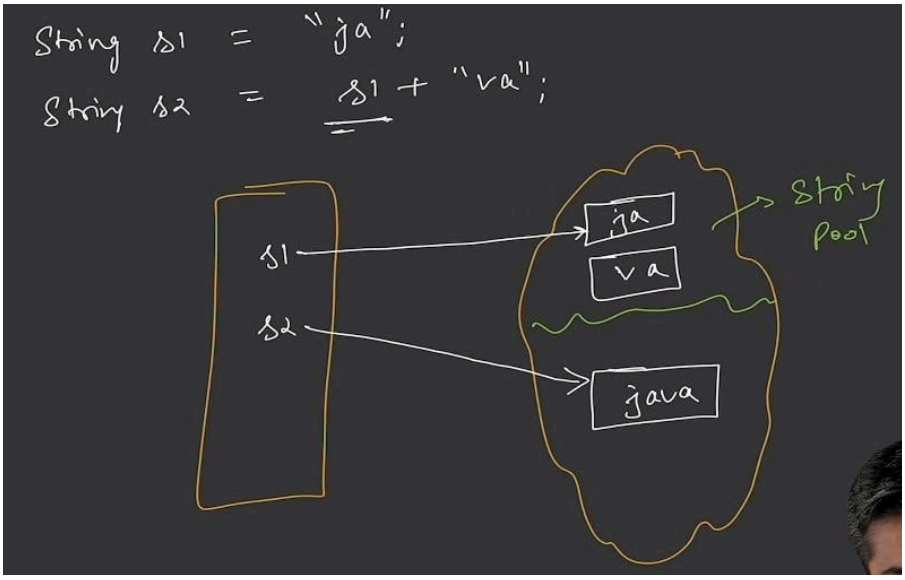
```java
public class StringPool {
    public static void main(String[] args) {
        String s1 = "Hello"; //literals objects
        String s2 = "Hello";
            System.out.println(s1==s2);//True-->kuki dono ek hi object ko reffer kr rhe ha.
        String s3 = new String ("Priyansh");
        String s4 = new String ("Priyansh");
            System.out.println(s3==s4);// false--> kuki dono alag-alag objects ko reffer kr rhe ha.
    }
}
```

Golden Rules: for String pool.

- Only compile time constants go to String pool automatically.



- Runtime created String go to heap.




```java
public class GoledenStringRule {
    public static void main(String[] args) {
        String s1 = "ja" + "va";// compile-time pr ho ta ha.StringPool memory me bnega.
        String s2 = "java";
            System.out.println(s1==s2); //True--> same object ko refrence kr rhe ha
        String s3 = "ja";
        String s4 = s3 + "va"; // run-time pr ho rha ha."s3" heap memory me bnega
            System.out.println(s3==s4); // false--> alag objects ko refer kr rhe ha.
        String s5 = "java";//compile-time pr hi ho ri ha kuki assignment operator ha isme.
        String s6 = s5;
            System.out.println(s5==s6);//true --> pointing same reference in Stringpool.
        String s = "Hello";//String pool me store hoga.
               s = "World";// s ab world ko point krega.
            System.out.println(s);// World is answer.
    }
}
```

Problem of immutability: **it wastes alot of memory.**

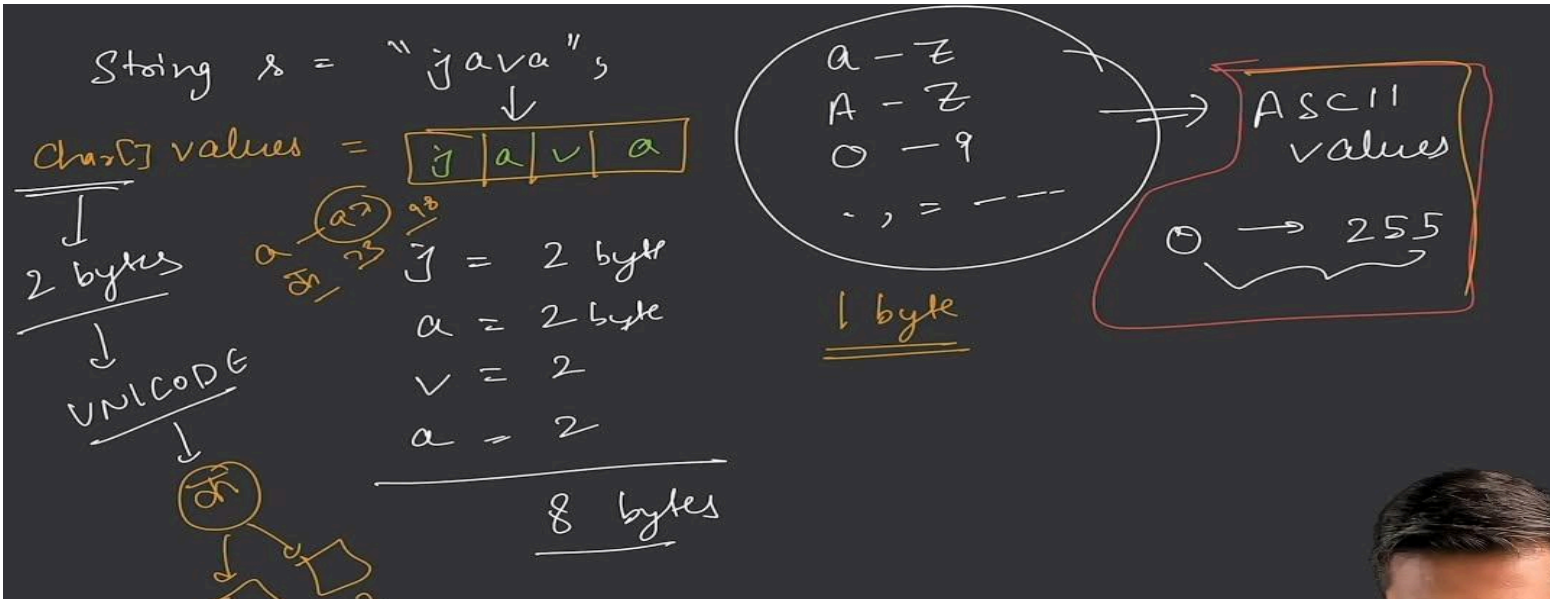
```java
public class ImmutablilityProblem {
    public static void main(String[] args) {
        // Problem of immutability.
        String s = ""; // /only this will store in string pool.
        for (int i=0; i<5; i++)  {
            s+=i;
            System.out.println(s);// "0"->"01"->"012"->"0123"->"01234", these all will store inheap memory seperatly.
        }
    }
}
```

- **Java 9 se phle String class me internally values char[ ] array me store hoti thi.**
- **Ab values byte[ ] array me store hoti ha.**

Reason→

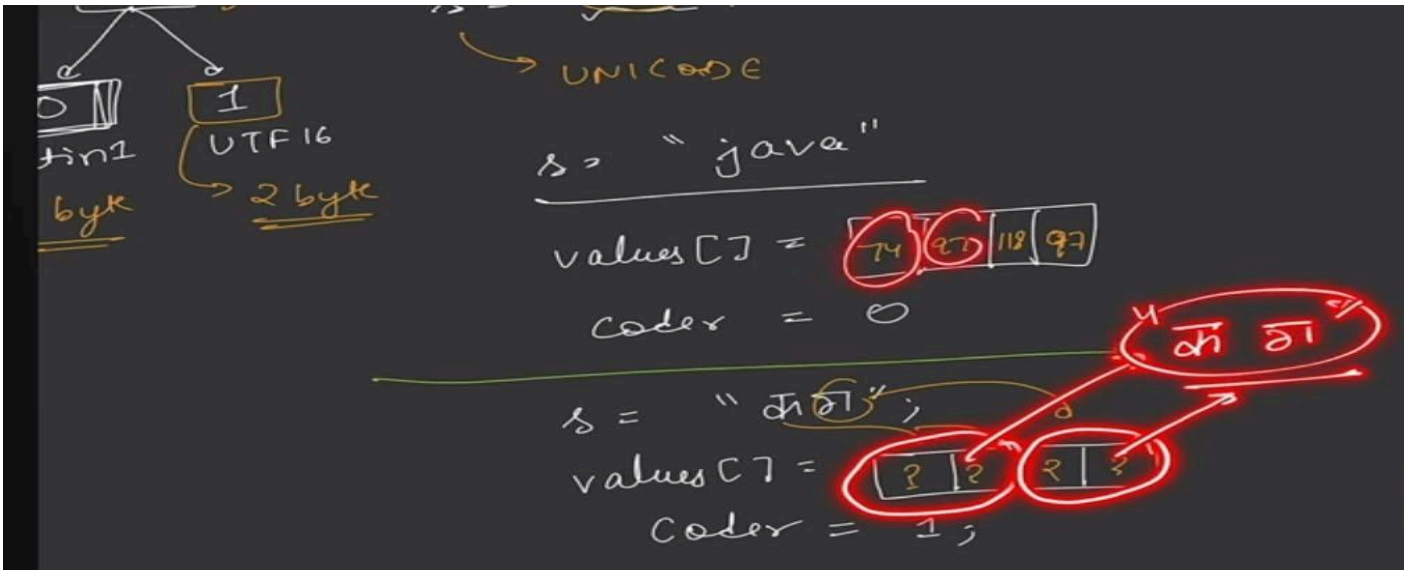
- **String me commonly use hone wale alphabets, symbols, etc ASCII-values ke hote ha.**
- **Or standard ASCII table me 128 characters hote ha,**
- **Or 1 byte total 256 alag-alag values ko store kr skta ha.**
- **Isliye ASCII ka koi bhi chracter 1byte me fit ho jata ha,**

- **Or 1 char ke ander 2bytes hoti jo total 65,536 alag-alag values ko store kr sktal ha.**
- **Or hume itni faltu storage waste krne ki zarurat nhi ha.**
- **Tabhi string values ko java 9 ke baad char[ ] se byte[ ] me change kr diya.**



Byte Coder:

- Iski sirf **2** hi values hoti ha:--> **0, 1**.
- Byte coder string class ko ye batati ha ki agar value0 ha to ek bar me ek 1character hi read krna , or agar value1 ha to ek bar me 2 characters ko read krna.
- **0** value tab hoti ha jab string ke ander ASCII characters ki values dalne wale hote ho, or iska naam diya ja **LATIN9** format, ye leta ha **1byte character**.
- **1** value tab hoti ha jab string ke ander ASCII characters ki limit ke bhar jate ho, iska naam ha **UTF16** format, ye leta ha **2bytes characters.**.

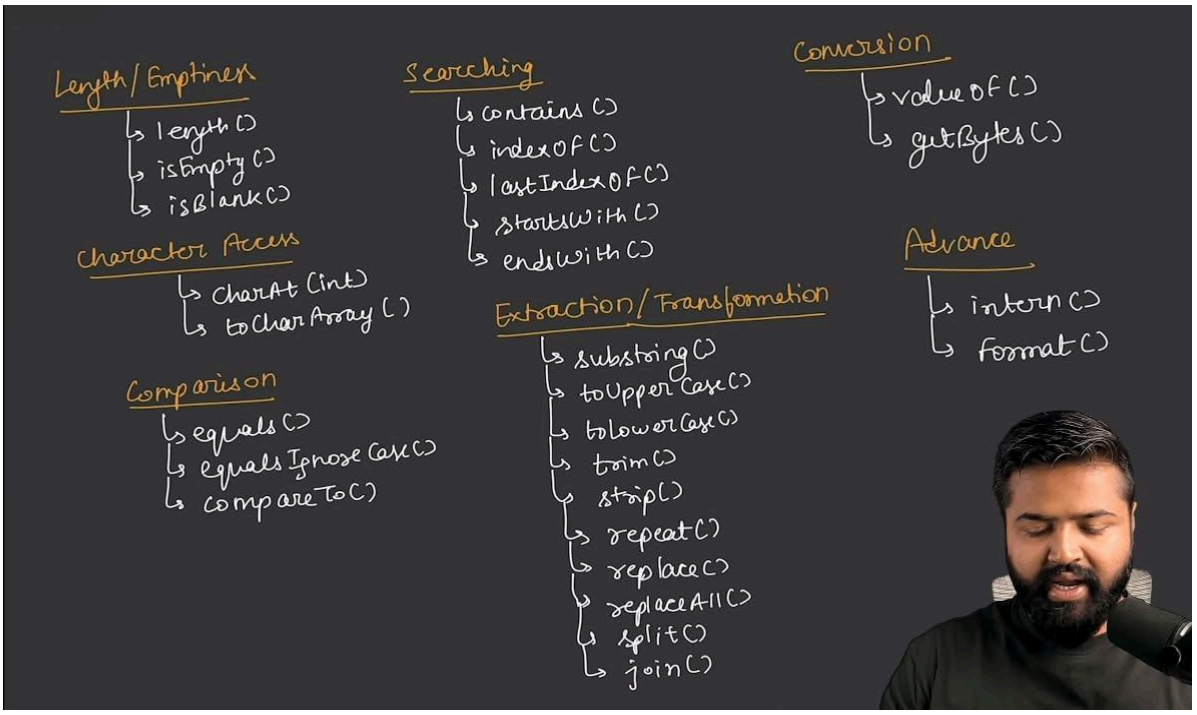


Strings - DeepDrive / Part-2

Internally String class→

```
Public final class String {
                                private final byte[ ] values;
Instance variables→             private final byte coder;
                                private int hash; }
```

Important methods in string class→




```java
String s1 = new String ("Priyansh");
String s2 = new String("Priyansh");

// Length /Emptiness.
    System.out.println(s1.length());//8
    System.out.println(s1.isEmpty());// false--> agar sirf "spaces" hoge to bhi ye false hi hoga.
    System.out.println(s1.isBlank());//false---> agar sirf "spaces" hoge to ye true hoga.
// Character accrss.
    System.out.println(s1.charAt(2));//"i"--> index value of the 2nd index.
    System.out.println(s1.toCharArray());// Priyansh--> char[] arr = s.toCharArray();
// Comparison Methods.
    System.out.println(s1.equals(s2));//true--> case sensitive.
    System.out.println(s1.equalsIgnoreCase(s2)); //true--> it ignore the cases of letters.'A','a'
    System.out.println(s1.compareTo(s2));/* it performs lexicographical comparison-->
    a dictionary comparison, matlab--> "saurav" / "sairav" --> "sairav" phle aayega.
    -> this method=>gives -ve[s1<s2], 0[both equal], +ve[s1>s2] outputes dega.
    ->s1 or s2 ki ASCII values ko add krke totals ko subtract kr dege.              */
// Searching.
    System.out.println(s1.contains("ansh"));// true--> it is case sensative.
    System.out.println(s1.indexOf("y"));//3-->index value.
    System.out.println(s1.lastIndexOf("y"));// repeated text me last text ka index.
    System.out.println(s1.startsWith("Pr"));//true-> case sensative.
    System.out.println(s1.endsWith("sh"));//true-> case sensative.
// Extraction / Transformation.
    System.out.println(s1.substring(1, 4));// "riy"=>isme endIndex "4"excluded hota ha.
    System.out.println(s1.toUpperCase());//PRIYANSH
    System.out.println(s1.toLowerCase());//priyansh
    System.out.println(s1.trim());//it trims only spaces from start and last, only for ASCII values.
    System.out.println(s1.strip());// same as trim(); but UNICODE friendly.
    System.out.println(s1.repeat(4));//repeats Priyansh 4 times.
    System.out.println(s1.replace('i', 'p'));//Prpyansh
    System.out.println(s1.replaceAll("ri", "uz"));// jitne bhi "ri" hoge sabhi replace ho jayge.
// Split Method-->it splits Arrays.
    String s3 = "Ram, Raghav, Saurav, Rinku, Yash";
        String[] arr = s3.split(",");
            for(String s : arr){
                System.out.println(s);// sabhi names alag-alag line me aayege.
            }
//join method=> it is static method.
```

```
        System.out.println(String.join("-", "a", "b", "c"));//a-b-c
    // Conversion. from int--> String.
        String s4 = new String(String.valueOf(10));
            System.out.println(s4);//"10"
    // to get string's UNICODE values.
        byte[] arr1 = s1.getBytes();
            for(byte i : arr1){
                System.out.print(i + "-");// sabki UNICODE values aayegi.
            }
    // Advance method. intern();
        String s5 = new String("Hey Ram");
        String s6 = s5;
        String s7 = s5.intern();// ye s5 ki value ko heap memory se String pool wali memory me copy krega.
    // Advance method. format(); it is Static method.
        String name = "Priyansh";
        int age = 28;
            System.out.println("Hello "+ name + ", your age is " + age );// but this is not readable.
            System.out.println(String.format("Hello %s, your age is %s", name, age));
```

Constructor overload Methods in String

```
String s1 = new String();// Empty string.
      String s2 = new String(""); // also same empty String.
          System.out.println(s1);         System.out.println(s2);
      String s3 = new String("Hello");//ye heap me store hoga, or iski ek copy String pool me hogi.
          System.out.println(s3);
                                        // Char Array--->
      char[] arr = {'A', 'd', 'i', 't', 'y', 'a',};
      String s6 = new String(arr); // to print all array.
          System.out.println(s6);//ans-->Aditya.
                                         //char Array Subset--->
      // offset-->Array ki place value, 1st array block is always at 0th place.
      // count-->counting, always starts from 1, 2, 3....
      String s7 = new String(arr, 0, 4);//Ans--> Adit-> from 0th array aount 4 digits.
          System.out.println(s7);
                                          // byte Array subset-->
      byte[] arr2 = {94, 95, 96, 97};
      String s8 = new String(arr2, 3, 1);
          System.out.println(s8);
                                          //StringBuilder-->
      StringBuilder sb = new StringBuilder("Hello");
      String s9 = new String(sb);
          System.out.println(s9);// result--> Hello.
                                          // StringBuffer-->
      StringBuffer sbf = new StringBuffer("bye yr");
      String s10 = new String(sbf);
          System.out.println(s10);
```

StringBuilder / StringBuffer

- **Both are mutable, it means it can change things in same string.**
- **These both are the classes, and these inherits from parent class " AbstractStringBuilder".**
- **These classes are also present in java.lang package, as same as String class.**
- **Both classes have same methods except,**
- **StringBuilder→ it is not thread safe.**
- **StringBuffer→ it is thread safe.**
- **StringBuilder is faster then StringBuffer.**
- **StringBuilder is used more then StringBuffer because StringBuffer is slower.**

StringBuilder/ StringBuffer :

- **Ye values ke array ko bada rkta ha.**
- **Default arry ki values capacity 16 hoti ha.**
- **Agar values capacity bhar jaye to fir vo (initial capacity * 2) + 2 formula use krta ha, to capacity ho jaygi → (16*2+2=34) ans.**
- **These, do not override the 'equals();'.**

StringBuilder class→ **Instance variables→**	**Class StringBuilder extends AbstractStringBuilder{** • **Byte[ ] values;** • **Int Count;** → ye count krta ha ki capacity me kitna space bhar gaya ha. **}**
constructor in StringBuilder/ StringBuffer→	• **new StringBuilder();** // empty constructor. • **new StringBuilder(50);** //default array capacity 50 ho jaygi. • **new StringBuilder("java");** // capacity=> 16 + 4= 20.
Methods in StringBuilder/ StringBuffer—>	• **append();** • **insert();** • **delete();** • **replace();** • **reverse();** • **charAt();** • **setcharAt();** • **length();** • **capacity();** • **Ensure capacity(100);** • **trimToSize()**;// empty spaces or capacity ko trim kr deta h jitna array ka size hota ha.

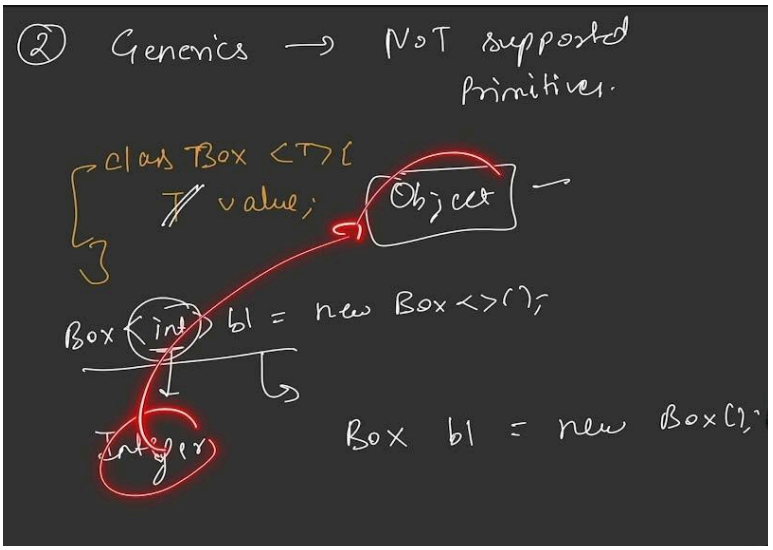
Java Generics

Java is typed language.

Type Safety→ example: we can't store string type value directly into integer datatype.

Primitives are not supported in Generics why?

Ans→



Upcasting & Downcasting

- **These are done in non-primitive.**

UpCasting→ specific class ko general class me convert kr dena, matlab sub-class ko super-class ke reference me Assign krna.

- Ye casting automatically ho jati ha, no casting keyword required.

```
    String s = "Hello";
        Object obj = s;
            System.out.println(obj);// Hello.
```

DownCasting→To convert super-class into the sub-class, keyword of sub-class required.

```
Object obj1 = "Priyansh";
        String s1 = (String) obj1;
            System.out.println(s1);// Priyansh
```

DownCasting exception→ Run-time exception:

```
Object obj2 = 19;
        String s2 = (String) obj2;
            System.out.println(s2);// ClassCastException--> Run-time exception
```

Problem before Generic→

- hume ya to har datatype **instance variable** k lia alag-alag class bnani pdegi ya fir same **class** me har type ke **instance variable** ke liya alag-alag **getters, settes, constructors** bnane pdege.
- Ya fir **object type** ka instance variable bnana pdega, pr usme **type information** lost ho jati ha, because it is **too generic** class.
- Or **object type instance variables** me hum har datatype ke liye main funtion me downcasting krni pdegi, or ye bhot **risky** ho jata ha, therfore using Object and make it too generic is not good & safe way.

```
public class ProblemBeforeGeneric {
    public static void main(String[] args) {
        Box b1 = new Box (10);
        Box b2 = new Box ("Hello");
        Box b3 = new Box (true);
            System.out.println(b1.getValue());// ans->10
            //System.out.println(b1.getValue() + 5);// error--> not sure.
                        // so we have to do--> DownCasting..
        Integer x = (Integer) b1.getValue();
        String s = (String) b2.getValue();
        Boolean b = (Boolean) b3.getValue();
            System.out.println(x + 5);
            System.out.println(s + 5);
            System.out.println(b);
    }
}
class Box {
    private Object value;
        Box(Object value) {
            this.value = value;
                //System.out.println(this.value);// constructor dalte hi print ho jaye.
        }
        public Object getValue(){
                //System.out.println(this.value);// main function me print statement na likhni pde.
            return this.value;
        }
        public void setValue(Object value){
            this.value = value;
        }
}
```

GENERIC CLASS→ye kisi bhi class ko kisi bhi type me kaam krne k lia access delati ha.

- **<T> ye parameter ha jiski madad se hm kisi bhi type m kaam kr skte ha.**
- **<T> ye methode me bhi same kaam krta ha.**

```
        Pair <Integer, String> p1 = new Pair<>(14, "Ram");
            System.out.println(p1.first + ", " + p1.second);
    }
}
class Pair < T, U > {
    T first;
    U second;
        Pair(T first, U second){
            this.first = first;
            this.second = second;
          }
```

Generic method→

```
public class GenericMethods {
    public static void main(String[] args) {
        Integer y = getResult(5);
            System.out.println(y);
        String s = getResult("Priyansh");
            System.out.println(s);
        printPair(true, "Priyanshu"); // here is type inference.
    }
    public static <T> T getResult(T v){    //<T> TYPE PARAMETER.
        return v;
    }
    public static <T, U> void printPair(T first, U second){
        System.out.println(first + ", " + second);
    }
}
```

How to set Restrictions in generic classes→

```
public class RestrictedGeneric {
    public static void main(String[] args) {

        Box <Integer> b1 = new Box();
            b1.value = 5;
            b1.printDouble();
    }
}
            //Bounds in generics-->
                // Upper bound.
class Box <T extends Number> {// 'Number' is parent class of all numerical classes,yaha pr  hm ye bhi kr skte ha-->
//        <T extends Number  & interface1, interface2, etc>{}
    T value;
        public void printDouble(){
            System.out.println(value.doubleValue());
        }
}
// Agar me chahta hu ki meri class bas har tarah ke numbers ke sath kaam kr ske
//  or 'string', 'boolean', etc ko restrict kr de.
```

<-- WildCards <?> -->

- **<List>** is a **interface** in java, it is like a Array.
- Ther is a **concrete** class that inherits the **<List> interface→class ArrayList.**
- **Both <List> & <ArrayList> are in java.util package, we have to import them.**

ArrayList object→ List<Integer> list = new ArrayList<>();

—----------PROBLEM BEFORE **WILDCARDS**—----------

Note→

If class Dog extends Animal{};

List <Dog> dogs = new Arraylist<>();

List<Animal> animals = dogs;

This is not allowed, because Generics in java brakes the parents child relationship.

Therefore, Generic <A> **is not subType of** Generic <B>.

Reason: Type safety will break.

Therefore, Generic is invariant in java→

- Even if A is child of B.
- **Generic<A>** is not and never child of **Generic<B>**.

```
import java.util.ArrayList;
import java.util.List;

public class ListGenerics {
    public static void main(String[] args) {
        //Invariant in Generics-->
                                // Animal animal = new Dog();
                                //     animal.eat();
                                //     animal.walk();
                                // //animal.bark(); this action can't be perform.
          //List <Dog> dog1 = new ArrayList<>();
          //List <Animal> animals1 = dog1; this is not allowed.
                                Dog[] dog2 = new Dog[10];
                                Animal[] animals2 = dog2;
                                animals2[0] = new Animal();
                                    for (Animal animal : animals2){ // for every Animal animal in animals2
                                        animal.eat();
                                    } // this is also not allowed.
    }
}
class Animal{
    void eat(){
        System.out.println("Eating");
    }
    void walk(){
        System.out.println("Walking");
    }
}
class Dog extends Animal{
    void bark(){
        System.out.println("Barking");
    }
}
```

WildCards→ <?>

- It is a **List <of any type>,** that can store anything**.**
- **<?>** it is a list of specific type but we dont know what ?
- parent/child relationship is not maintained.
- Known as invariant.
-

Problems solved by WILDCARD<?>

```
import java.util.List;
import java.util.ArrayList;
```

```
public class WildCards {
    public static void main(String[] args) {
        // Without WILDCARD.
                    // List <Dog> dogs = new ArrayList<>();
                    //     dogs.add(new Dog());
                    //     dogs.add(new Dog());
                    //    fun(dogs);// not possible kuki me dogs ki list Animals ki list
                                //  me pass nahi kr skta.
                    // List <Animal> animals = new ArrayList<>();
                    //     animals.add(new Animal());
                    //     animals.add(new Animal());
                    //         fun(animals);// this is possible and Eating/Eating is answer.
        // With WILDCARD.
        List<Animal> animals = new ArrayList<>();
            animals.add(new Animal());
            animals.add(new Animal());
                fun(animals);// ANS--> 2-times-->"Animal"
        List<Dog> dogs = new ArrayList<>();// now this is possible with WILDCARD.
            dogs.add(new Dog());
            dogs.add(new Dog());
                fun(dogs);// ANS--> 2-times-->"Dog"
    }
    // Without WILDCARD.
                    // static void fun(List<Animal> animals){
                    //     for(Animal animal : animals) {
                    //         animal.eat();
                    //     }
                    // }
    // with WILDCARD.
    static void fun(List<?> values) {
        for (Object obj : values) {
            System.out.println(obj.getClass().getName());
        }
    }
}
class Animal{
    void eat() {
        System.out.println("Eating");
    }
    void walk() {
        System.out.println("Walking");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

WildCards with bounds→

- Wildcards can be only used with **lists** and with **generics** that brake **parents / child** relationship.

? extends→ it is for reading and known as **covariant**.

? super→ it is for writing and known as **conter-variant**.

PECS (producer = extends, consumer = super) rule.

```
import java.util.ArrayList;
```

import java.util.List;

```
public class WildCardsBounds {
    public static void main(String[] args) {
        // WildCard with "upper bound"---> "? extends"
```

```
            List <Dog> dogs =  new ArrayList<>();
                dogs.add(new Dog());
                dogs.add(new Dog());
                    fun(dogs);//--> Dog is Eating, Dog is Eating
            List <Animal> animals = new ArrayList<>();
                animals.add(new Animal());
                animals.add(new Animal());
                    fun(animals);//--> Animal is Eating, Animal is Eating
        // WildCard with "Lower Bound"---> "? super"
                List <Animal> animals2 = new ArrayList<>();
                    animals2.add(new Animal());
                    animals2.add(new Animal());
                        fun2(animals2);

    }
    // WildCard with "upper bound"-->(extends).
        static void fun(List <? extends Animal> values){
            for(Animal a : values) {
                a.eat();
            }
        }
    // WildCard with" <? super>
        static void fun2(List <? super Animal> values){// minimum (Animal) class can be added.
            values.add(new Animal());
            values.add(new Dog());
            values.add(new Cat());
                for(Object obj : values) {
                    Animal a = (Animal) obj;
                        a.walk(); //Ans---> 5 times "walking"
                }

        }
}
class Animal {
    void eat() {
        System.out.println("Animal is Eating");
    }
    void walk() {
        System.out.println("Walking");
    }
}
class Dog extends Animal{

    @Override
    void eat() {
        System.out.println("Dog is Eating");
    }
     void bark() {
        System.out.println("Barking");
    }
}
class Cat extends Animal{
}
```

Type Erasures

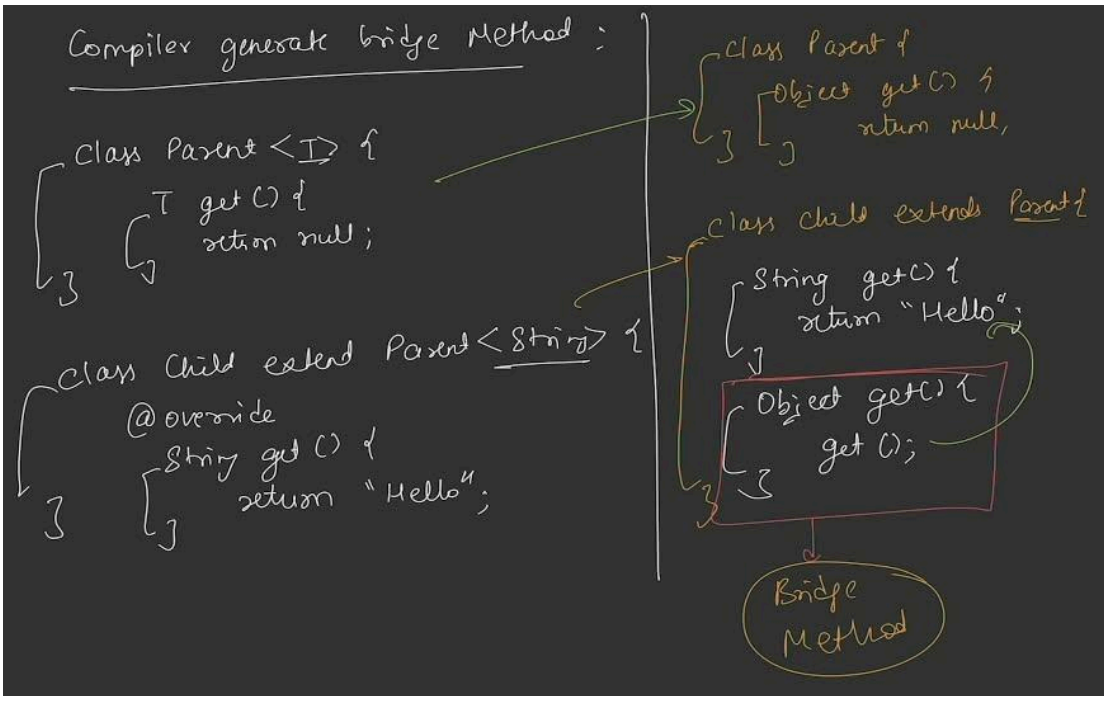
Does java knows Generics?---> no it dosen't know, it gets changed in the following things.

Rules of removing Generics during Run-Time→

If no bound replace with Object.	class Box <T>{T value;} →	class Box {Object value;}
If pairs no bound replace with Object	class Box <T, U>{T first; U second} →	class Box {Object first; Object second;}
If bounded replace with the bound, "extends"	class Box <T extends Number> { t value;} →	class Box { Number value;}
Insert casting automatically	Box <Integer> b1 = new Box<>(5); Integer x = b1.value;	Box b1 = new Box(5); Integer x = **(Integer)** b1.value;

Therefore we can't perform some actions with Generics→

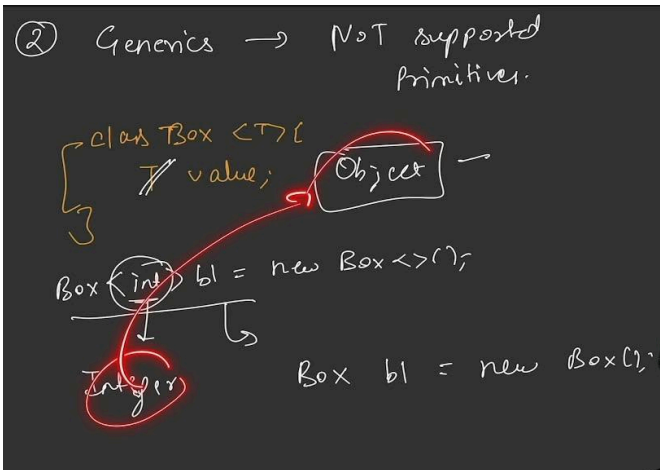
- (instance of) operator can't be used in Generics.
- Methods cant be overloaded on the basis of Generics.
- Compiler generated bridge method:



Why type Erasuer? → just to maintain the run time of the old version of codes, so that entire run-time execution remains unchanged.

Primitives are not supported in Generics why?

Ans→ because they can't be replaced by Objects.



Collection Framework introduction

In coding we are dealing with code that is stored in object inside (object oriented Programming). Taki hum alag-alag QUERY'S kr ske.

DATASTRUCTURES→

- The need of **DataStructure** is for storing **data** efficiently and also to perform **queries** on the stored data.
- If there is a Arry of numbers, [1, 23, 5, 45, 7,87,76,43,5,0]--> this **datastructure** is not organized.
- And if we have **query** to get the biggest number→therfore, in **time complexity** muj laga **Order of (N)** time. Or ye query mushkil ho gi.
- Therefore, **organized datastructure** for the above **query** is→ array in ascending order, and we will get the answer in **constant time**→ order of (1).

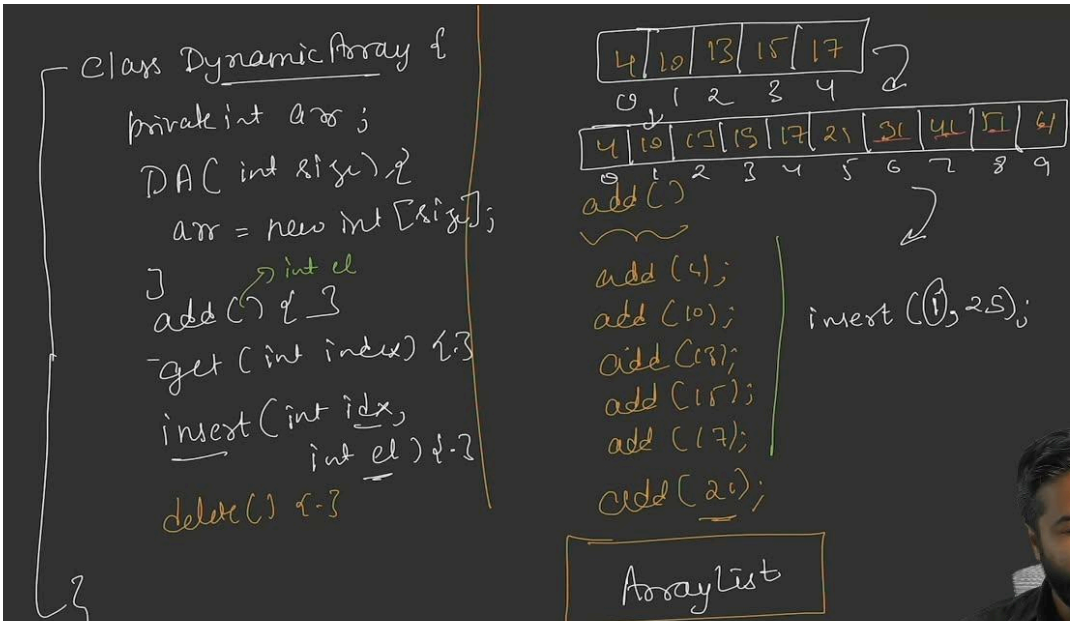
Therefore, kisi query ko easily access krne ke liye organized datastructures ka framework ko collection framework khte ha.

- Java provides support in general all datastructers, by giving classes.

Data Structures Types

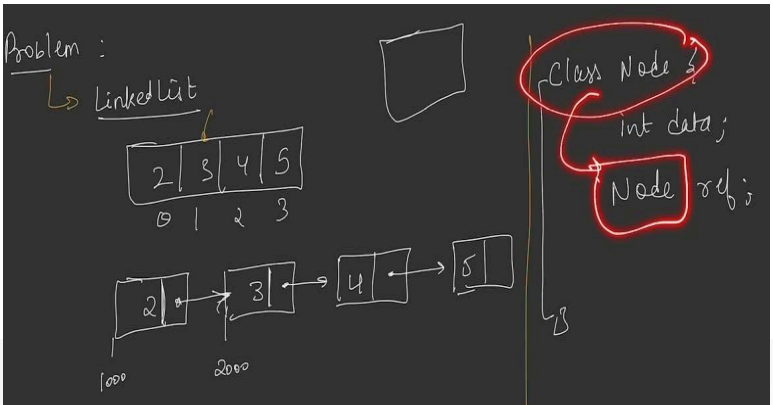
Dynamic Array DataStructure:

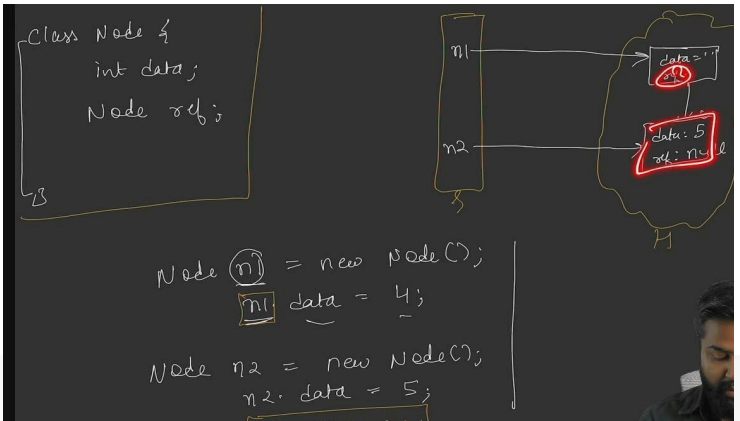
- isme Array ka koi fixed size nahi ha, size demand ke hisab se extend hota ha.
- Dynamic Array ke saare kaam ArrayList me ho skte ha jo java support krta ha.



Linked list DataStructure:

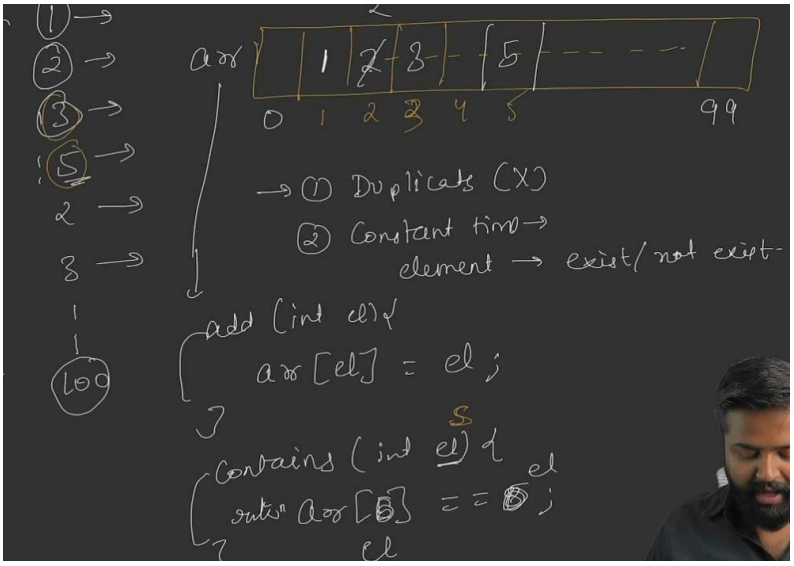
- It dosen't store data in the **contiguous memory address** like **Array.**
- It stores the reference of the next value of same node, iska storage kbhi full nahi hoga.





Set DataStructure:

- Duplicates are not allowed in this dataStructure.
- Tell in constant-time that element exists or not.
- This is possible by the dataStructure here→
- This is with particular range of array.
- For without range Array it is done with modulus method.



Stack & **Queues** DataStructure**:**

Stack:

- It means ki jis order me **data** Array me dala ha, uske **opposite** order me data ko nikalna ha.
- Iske entry or exit points **same** hote ha.
- It is also known as **LIFO→ Last in, First out,** in programming terms.
- Isme hume Array ke koi bhi access ni milte ha, taki hum use bich se access na kr ske.
- Only add(); & remove(); methods should be given.

Queues:

- It means ki jis order me **data** Arrray me dala ha, us hi **order** me data ko nikalna ha.
- It is also known as **FIFO→ First in, First out,** in programming terms.
- Isme Arrays ke entry & exit points alag-alag hote ha.
- Entry samne se or exit piche se hoti ha.
- Only add(); & remove(); methods should be given, same but with different style.

Maps DataStructure:

- It is as similar as Set DataStructure,
- .bs isme values ke sath proper keys hoti hai, hum pairs me store krte ha.
- Example→ Rohit, 101 / Aditya, 102.
- No duplicate values and keys are not allowed.

Tree DataStructure:

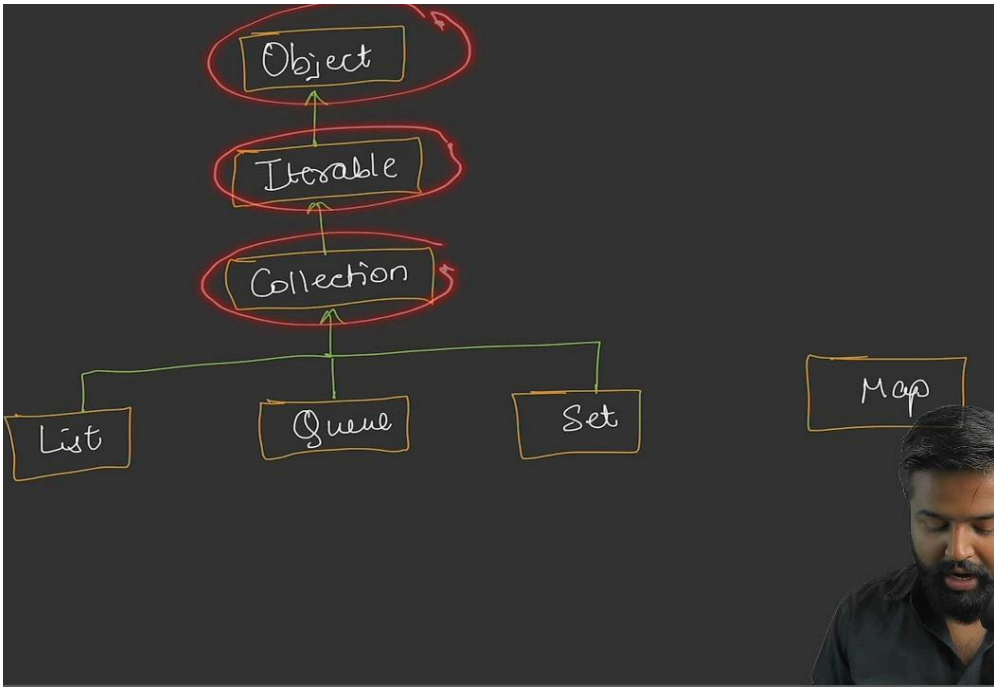
- **BST: Binary Search Tree→**
- 1node only have 2 childs.
- Left child is smaller then node.
- Right child is greater then node.

Java has given 2 classes for BST tree DataStructure.

- Class treeSet
- Class TreeMap

Note:

- All DataStructure are in **hierarchy**.
- **ArrayList** and **LinkList** both are child of **interface List.**
- Now, **List, Set, queues, stack,** etc all these interfaces are the **child** of the **parent interface COLLECTION.**



- **Map is hierarchy ki bahar ha kuki ye key or value ke sath deal krta ha.**
- **Baki sabhi dataStructures values ke sath deal krte ha.**

—--Iterable interface—--

Iterable interface→It is the root interface that represents any object whose element can be traversed one by one.

There are 2 types of DataStructures:-

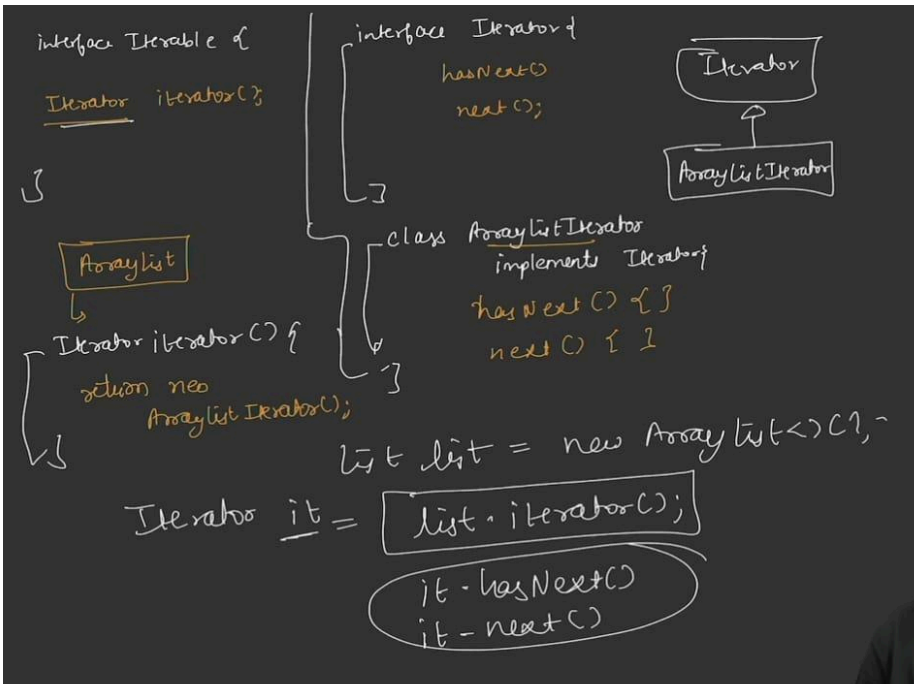
1) **Ordering maintaining DataStructures.**
- Only Dynamic ArrayList is index based.
  - ArrayList
  - LinkList

2) **Not maintaining Ordering DataStructures.**
  - Set
  - Map
  - Stack
  - Queues

```
import java.util.*;
public class IterablesIntro {
    public static void main(String[] args) {
        Collection <Integer> list = new HashSet<>();
            list.add(10);
            list.add(20);
            list.add(30);
            list.add(40);
            list.add(50);
                Iterator <Integer> it = list.iterator();
                    while(it.hasNext()){
                        System.out.println(it.next());
                    }
```

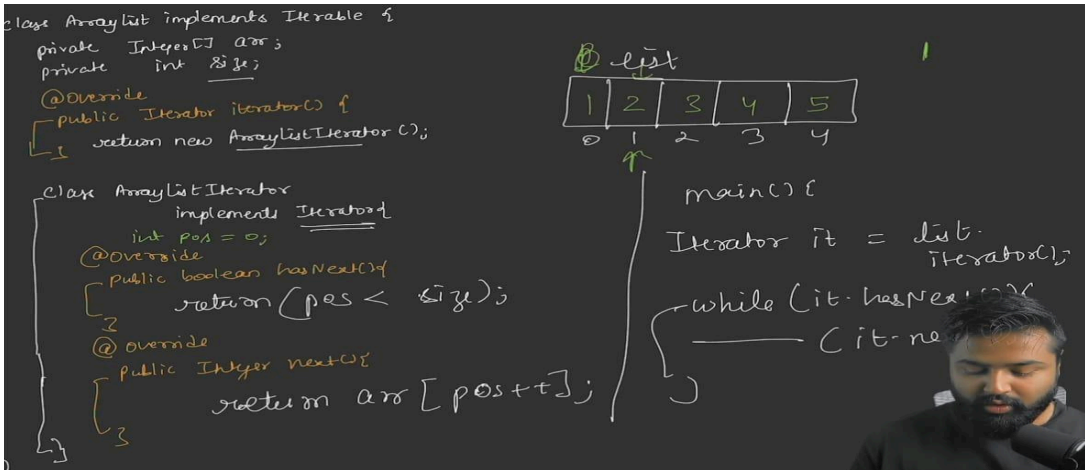
```
    }
}
// 'list.iterator();' this method is in List interface.
// Iterator<Integer> ka use isliy kr re ha taki 'hasNext()', 'next()'. mil sake.
```

Internally →

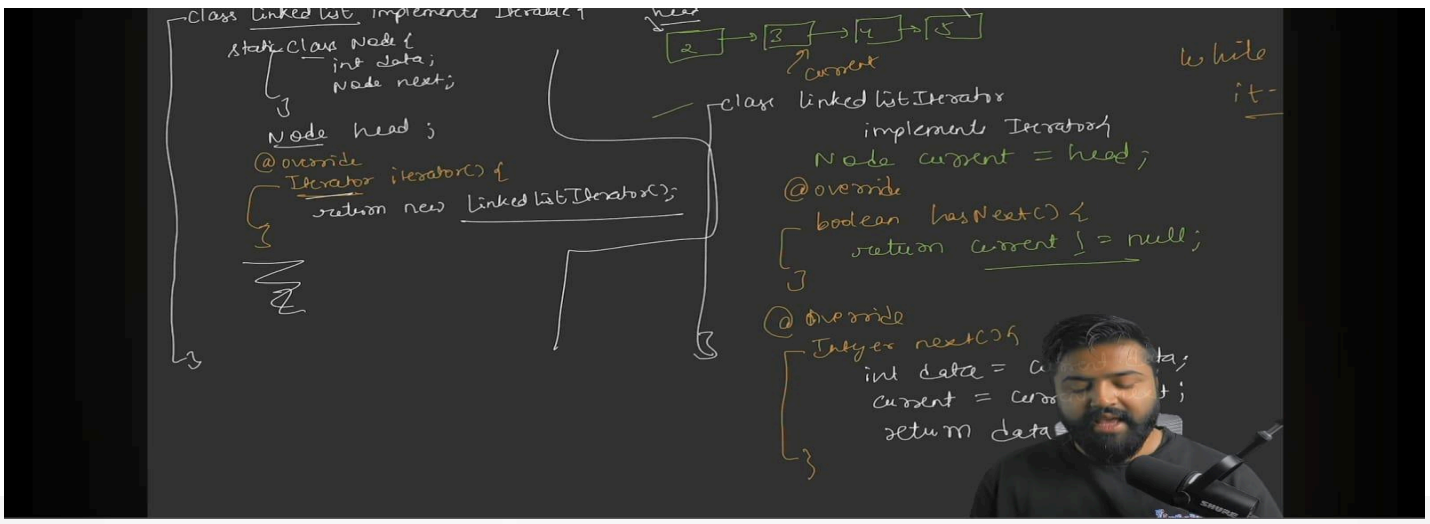


How Different **Collection** conceptually implements **ITERABLE / ITERATOR**

ArrayList:



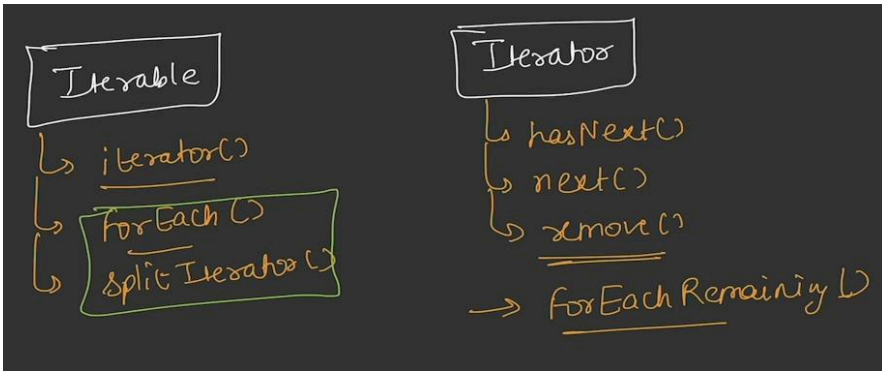
LinkedList:

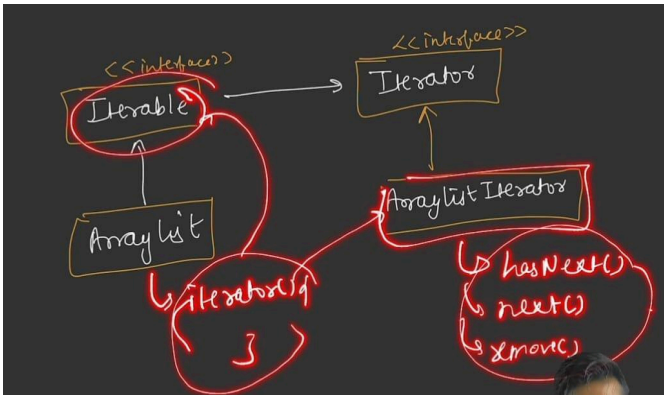


My own Iterable Interface

```java
import java.util.Iterator;

public class OwnIterator {
    public static void main(String[] args) {
        String[] names = {"Aditya, Rahul, Rohit, Saurav, Priyansh"};
        NameContainer container = new NameContainer (names);
            Iterator<String> it = container.iterator();
                while(it.hasNext()){
                    System.out.println(it.next());
                }
    }
}
class NameContainer implements Iterable<String>{
    private String[] names;
    private int size;
        NameContainer(String[] names){
            this.names = names;
            this.size = this.names.length;
        }
    @Override
    public Iterator<String> iterator(){
        return new NameContainerIterator();
    }
        private class NameContainerIterator implements Iterator<String>{
            private int position = 0;
                @Override
                public boolean hasNext(){
                    return position < size;
                }
                @Override
                public String next(){
                    return names[position++];
                }
        }
}
```

More concepts of Iterator→





—Collection Interface—

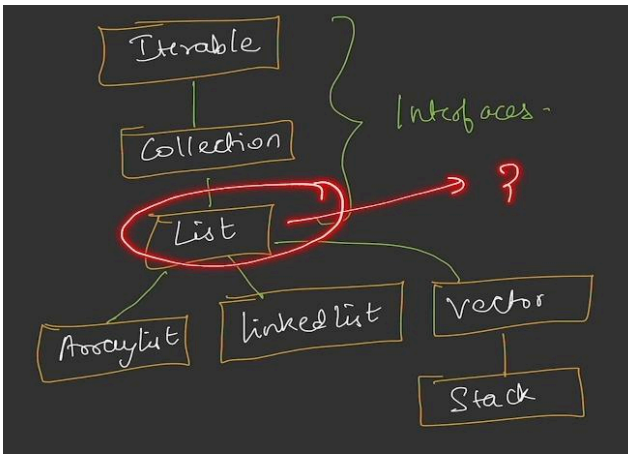
In java we have→ Two types of Collection→

Collection Interface:	Collections Class→ utility class.

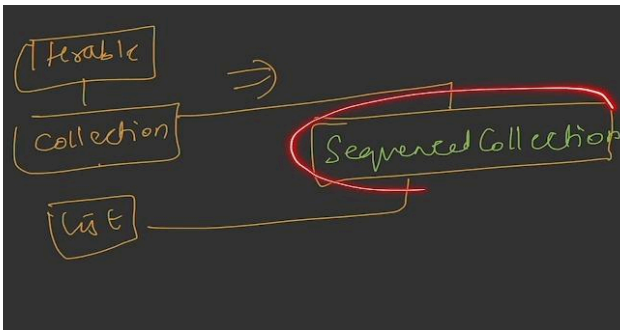
—Methods in collection Interface—

```java
// Conceptual View

public interface Collection<E> extends Iterable<E> {
    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator<E> iterator();
    Object[] toArray();
    <T> T[] toArray(T[] a);
    boolean add(E e);
    boolean remove(Object o);
    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? extends E> c);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    void clear();
    boolean equals(Object o);
    int hashCode();

    // modern java method
    default boolean removeIf(Predicate<? super E> filter)
    default Spliterator<E> spliterator()
    default Stream<E> stream()
    default Stream<E> parallelStream()
}
```

List Interface



- Elements have a position.
- Duplicates are allowed.
- Insertion Order is preserved.
- You can access elements by Index.



Methods in SequenceCollection Interface: after java 21

- getFirst()
- getLast()
- addFirst()
- addLast()
- removeFirst()
- removeLast()

List Interface specific Methods:

- **E get(index)**
- **set(index, E value)** // replace the value.
- **add(index, E value);** // shift the value.
- **boolean addAll(index, Collection<? Extends E>);**
- **remove(index);**
- **indexOf(Object o value);**
- **lastIndexOf(Object o value);**
- **listIterator();** // is iterator me hum bhot actions kr skte ha.
- **listIterator(int index);**
- **list.of();** // this method makes the list immutable.
- **copyOf();** // it makes the copy of the list.

Methods of ArrayList concrete class:

- **get(index i);** // order of 1times
- **set(index i);** // order of 1 times
- **add(i, 5);** // order of n times
- **remove(i, 5);** // order of n times

Methods only inside ArrayList class

- **ensureCapacity(100);** ensure kro ki iski minimum capacity 100 ho.
- **trimToSize();**

ArrayList is the most popular and most used list because→

- Random access at constant time.
- Cash friendly.
- Simple structure.

Constructors of ArrayList

- new ArrayList<>();
- new ArrayList<>(10);
- new ArrayList<>(Collection c);

LinkedList

- **get(index i);** // order of n times
- **set(index i);** // order of n times
- **add(i, 5);** // order of n times
- **remove(i, 5);** // order of n times

Set & Map Interfaces

- **Duplicates are not allowed.**
- **Constant time search operations.** It means it gives the result immediately and fast.

Drawbacks of Set & Maps

- **Positional access**→ values kisi order me store nahi hoti ha, kahi bhi store ho skti ha.
- It can't perform index based searches, like List DataStructures.

Maps

- It stores→ key, values.
- Duplicate **values** are allowed, but….
- Duplicate **keys** are not allowed.
- **Constant** time search.

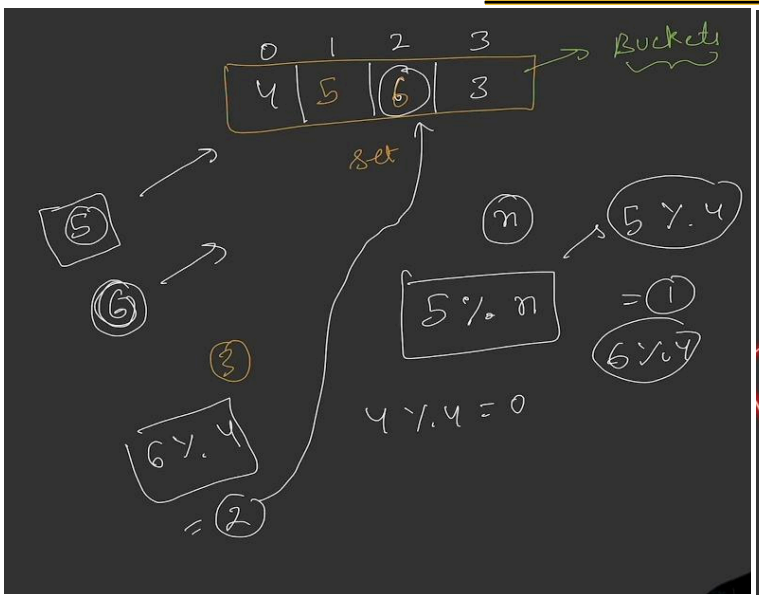
Example:

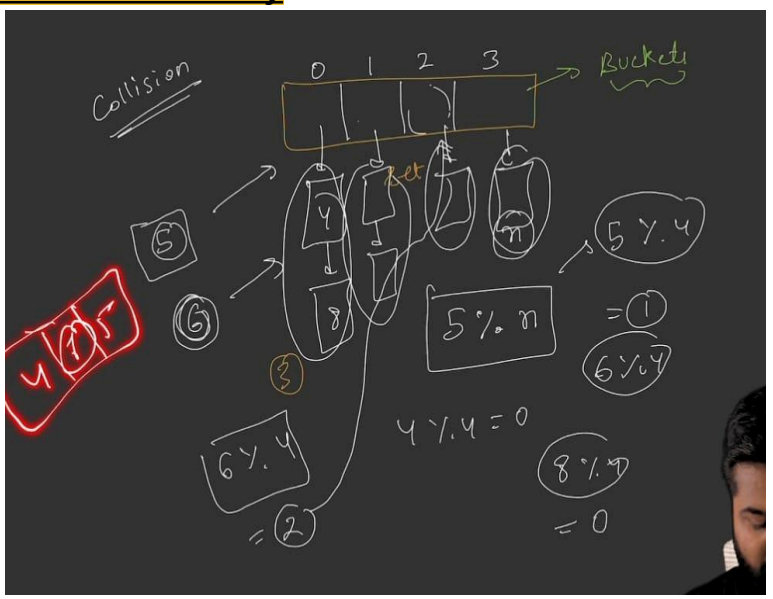
RollNumber and Students Names:--> here Student Names can have duplicate but RollNumbers can't be duplicates.

Methods of MAP INTERFACE

```java
import java.util.*;
public class SetandMaps {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
            set.add("Aditya");
            set.add("Rohit");
            set.add("Rohan ");
                System.out.println(set.contains("Ronit"));
        Map<Integer, String> map = new HashMap<>();
            map.put(100, "Rahul");
            map.put(101,"Aryan");
            map.put(102, "Gaurav");
            map.put(103, "Rahul");
                System.out.println(map.containsKey(101));
                System.out.println(map.get(102 ));
    }
}
```

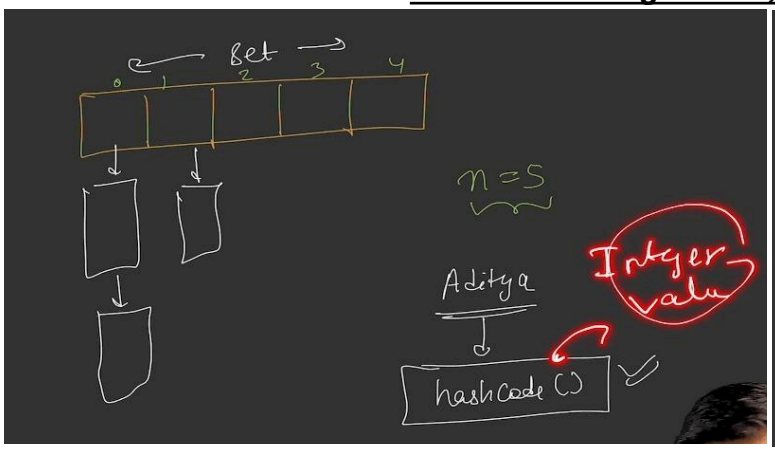
How Set values are stored internally

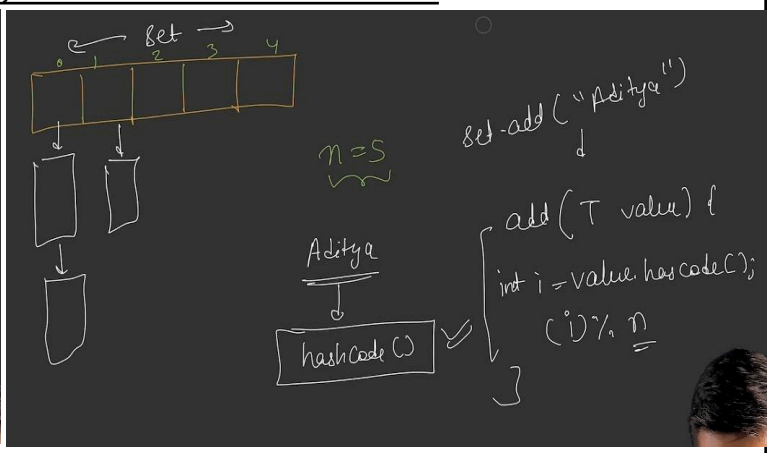




- **Kisi bhi value ko store krne ke lia set datastructure ke size ke sath module kr dete ha. Example→ 5%4=1 ans, it means 1st index.**
- **Or value ko nekalne ke liye us value ko fir se Set DataStructure ke size se module krana pdega, us se index value aaegi.**
- **Or ek hi index pr multiple values store krne ke liya, hum har index ko LinkedList me convert kr dete ha taki sabhi values particular index me save ho ske.**

In case of String values, Stored in Set DataStructure





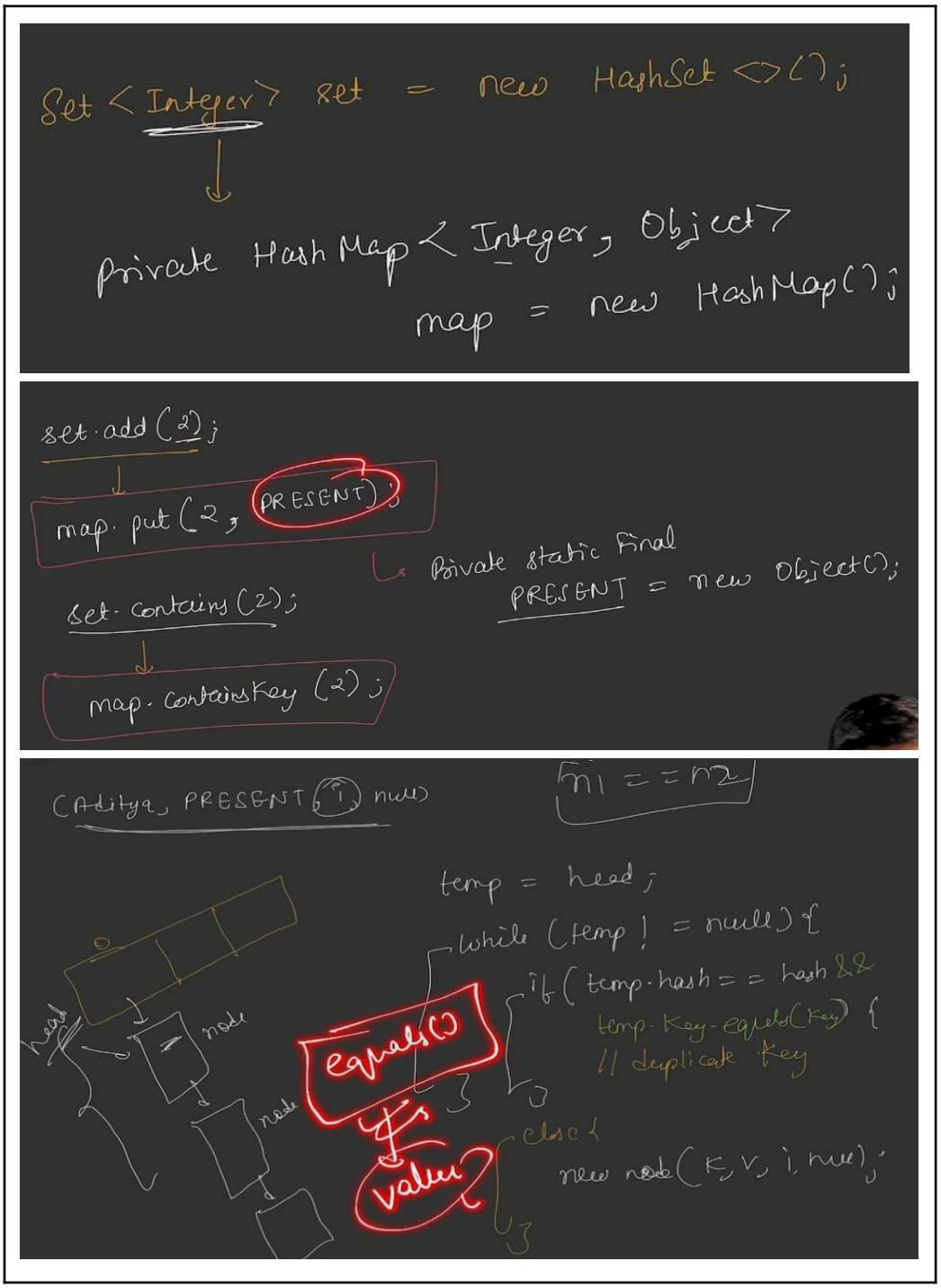
String values ka hum phle hashSet nekalte ha,
Or fir us hashSet ko set Datastructure ke size se module krte ha, taaki pta chle ki value kis bucket

me jani chahiye.
Or baki process same hi rhta ha..

How Map Values are Stored Internally

Maps me bhi values same hi store hoti ha.
Isme values key ka hash set nikal ke store hoti ha.
Fir hashSet ko DataStructure ke size se module krte ha, taki pta chle ki value kis bucket me jani chahiye.

Java me **set** nam ka koi dataStructure **nahi** hota, jab bhi hum set bnate ha to **internally** java ek **Map** hi bna ke deta ha.



- To handle **collision** in **Map** & **set buckets** we use **linkedList** and this process is known as **chaining**.
- One more way to handle collision but it is not used in java language, it is known as **OPEN ADDRESSING**.
- OPEN ADDRESSING→ isme hum kisi bhi bucket ki repeated values ko next bucket me shift kr dete ha. Or jo us next bucket ki original value ha use bhi next me shift krna pdta ha, & this is very complex.

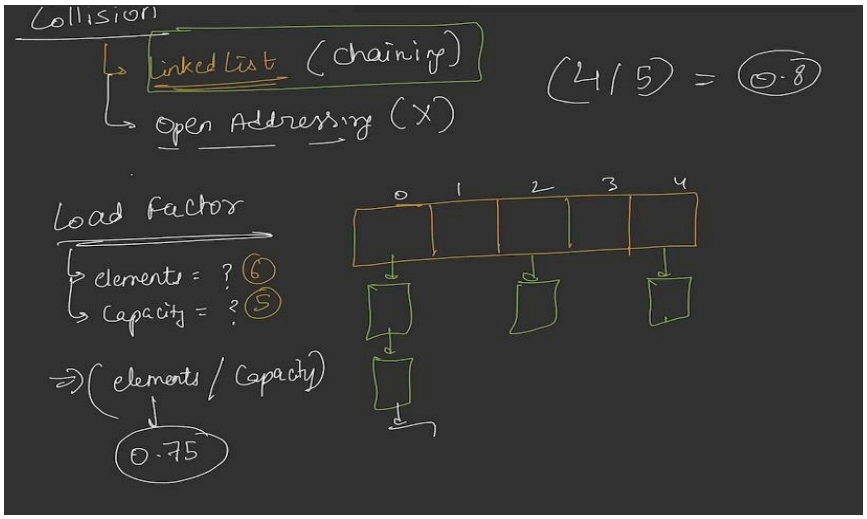
LOAD FACTOR in java

- How many number of elements ?
- How much capacity ?

We have a formula→ (elements / capacity)= if ans is >0.75 then it means ki hamare bucket ka array bhut full ha, it means 75% se zada full ha.

To us case me hum krege **Rehashing**→ ki hum existing array ko uske double size ke Array me dal dege.

- Isko krne ke piche ka simple reason ha, taki humari bucket me hume no of 1 times pr result mile, or load na pde.

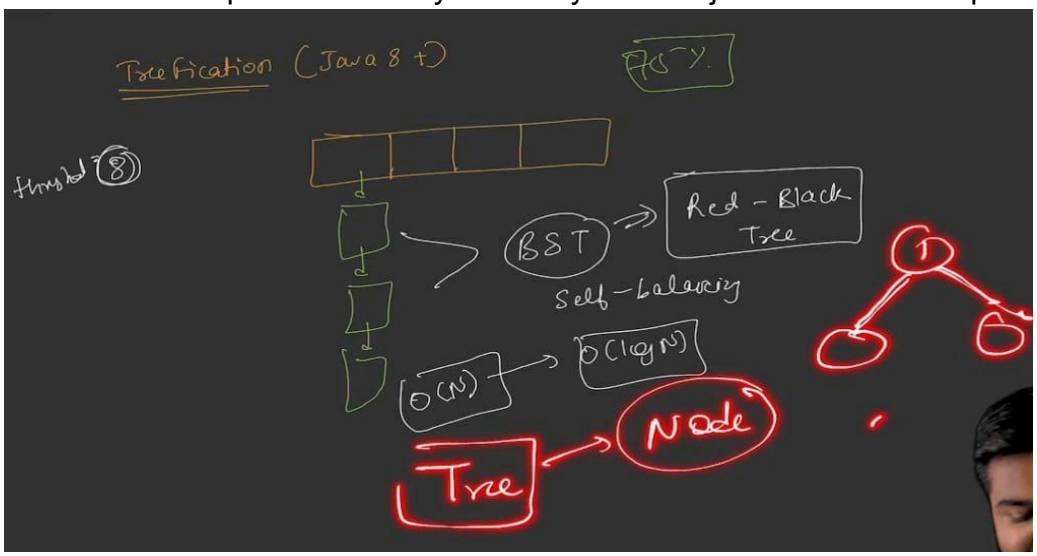


Steps of Re-Hashing.

- Create 2x as Array.
- Now again re-mod the keys of the array, and giving them new bucket indexings.
- Initial size of the array of Set & Map is 16 size, it means the initial capacity of the array is → 16.

Tree-fication (java 8+)

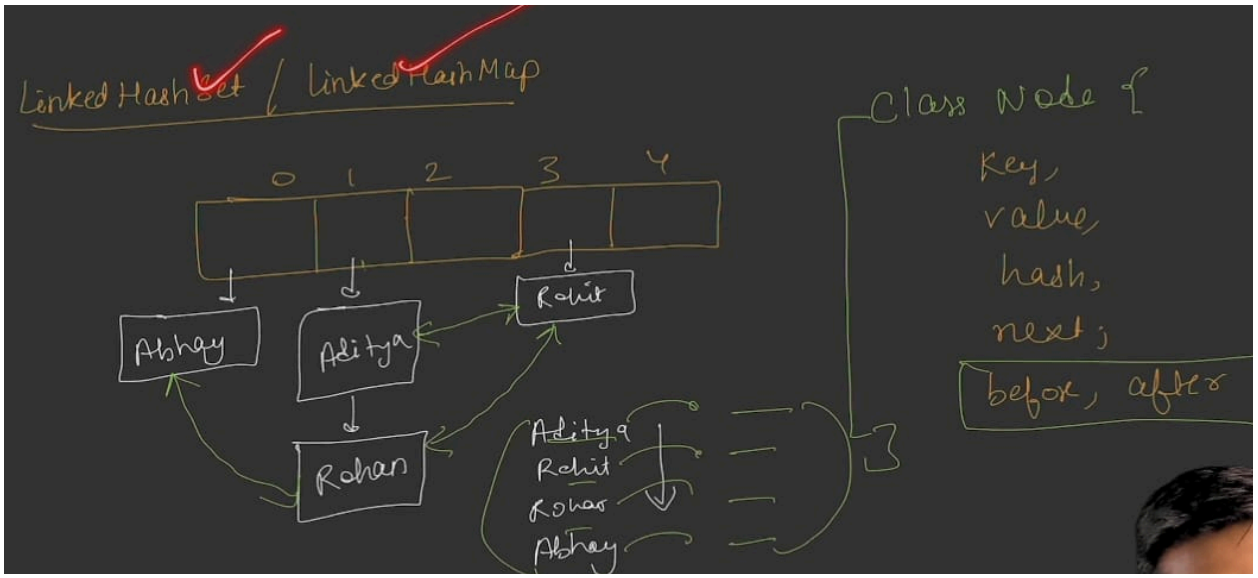
- Ye concept automatically internally hota ha jab bhi elements apni thrashold se zada hote ha.



- Kum elements ki collection ke liye hm prefer krte ha LinkedList or,
- Zada elements ki collection ke liye hum prefer krte ha Red-Black Tree.

Linked HashSet / Linked HashMap

- In me hume proper order milta ha.
- Doubly LinkedList is used in this.
- Is se dataStructure ka order preserved reheta ha.
- Ye thoda complex ha or sath hi ye storage zada leta ha.
- Iska use tabhi krna chahiye jab hume order preservation se matlab ho.



Tree-Map & Tree-Set

- Here we use (Self Balancing BST)--> (Red-Black Tree).
- Yaha pr hashMap wala concept kam nahi krta ha.
- Yaha pr hashing use nahi hoti ha.
- Yaha pr time complexity order of (log-n) ho jati ha.

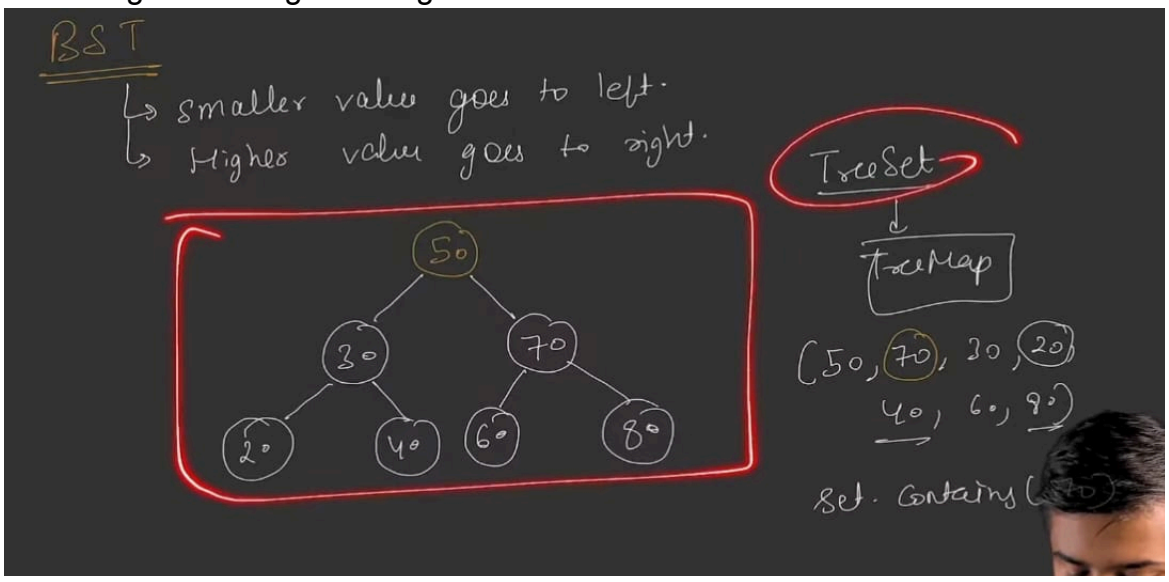
It unlocks many methods in dataStructure which can't be given by hashMap & Linked HashMap→

- Keys sorted order.
- Find the largest element.
- Find the smallest element.
- Range query…….give me numbers between 20 to 75.

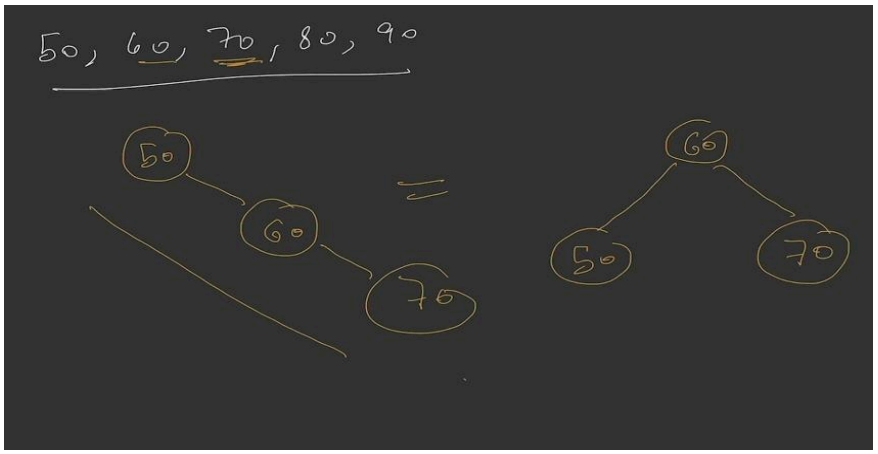
In case of strings these methods works on Lexicographical order→ dictionary order of Alphabets.

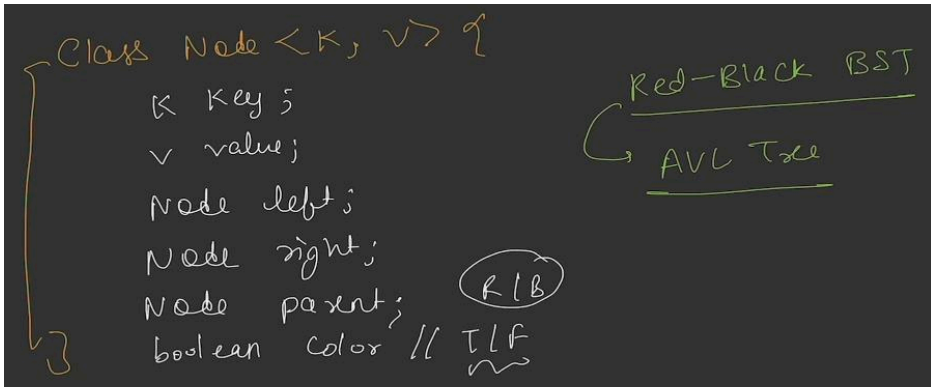
Concept of Normal BST

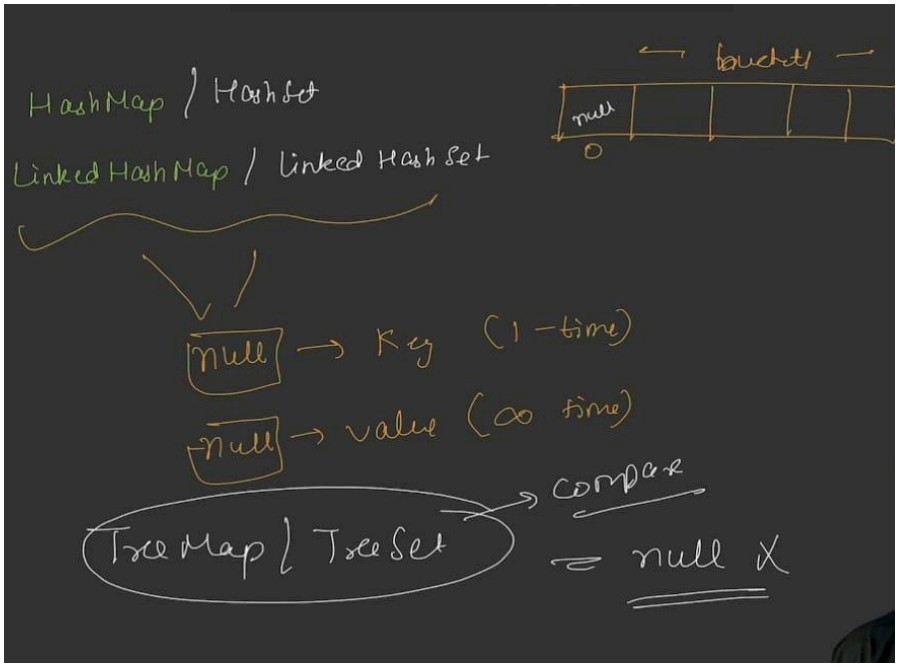
- Smaller value goes to left.
- Higher value goes to right.

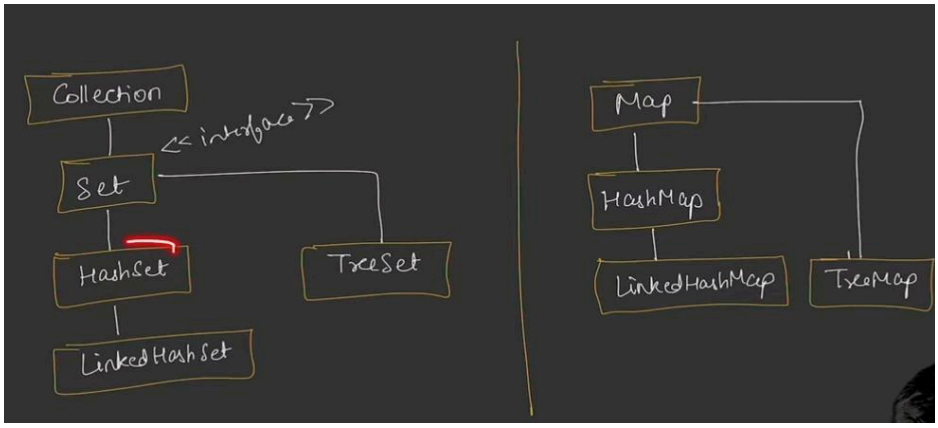


Concept of Self-Balancing BST

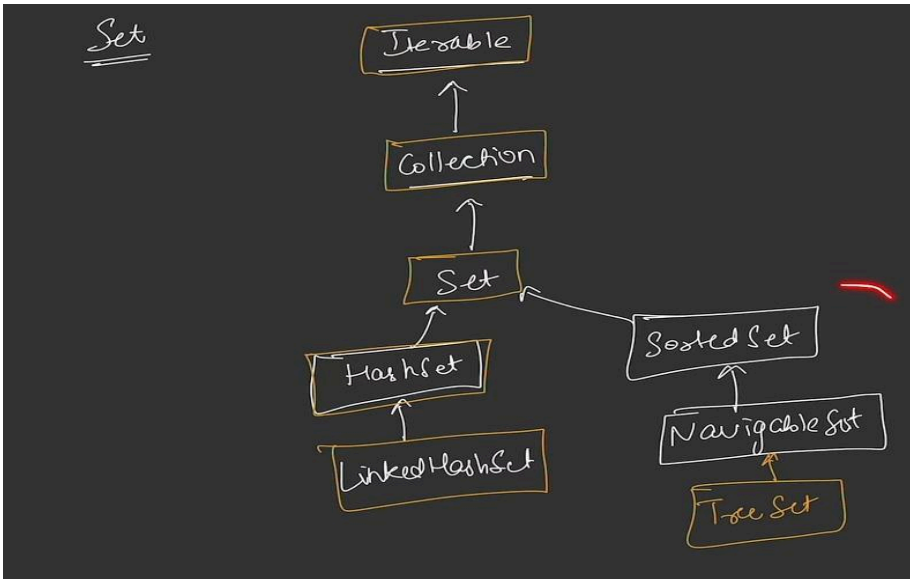

50, 60, 70, 80, 90
50
60
70
=
60
50
70


Class Node<K, V>{
K key;
V value;
Node left;
Node right;
Node parent;
boolean color // T|F
}
RIB
Red-Black BST
AVL Tree


HashMap / HashSet
LinkedHashMap / LinkedHashSet
bucket
null
0
null → Key (1-time)
null → value (∞ time)
TreeMap / TreeSet
compare
= null X


Collection
<<interface>>
Set
HashSet
TreeSet
LinkedHashSet
Map
HashMap
LinkedHashMap
TreeMap

Methods & Cunstroctors in Set & Map

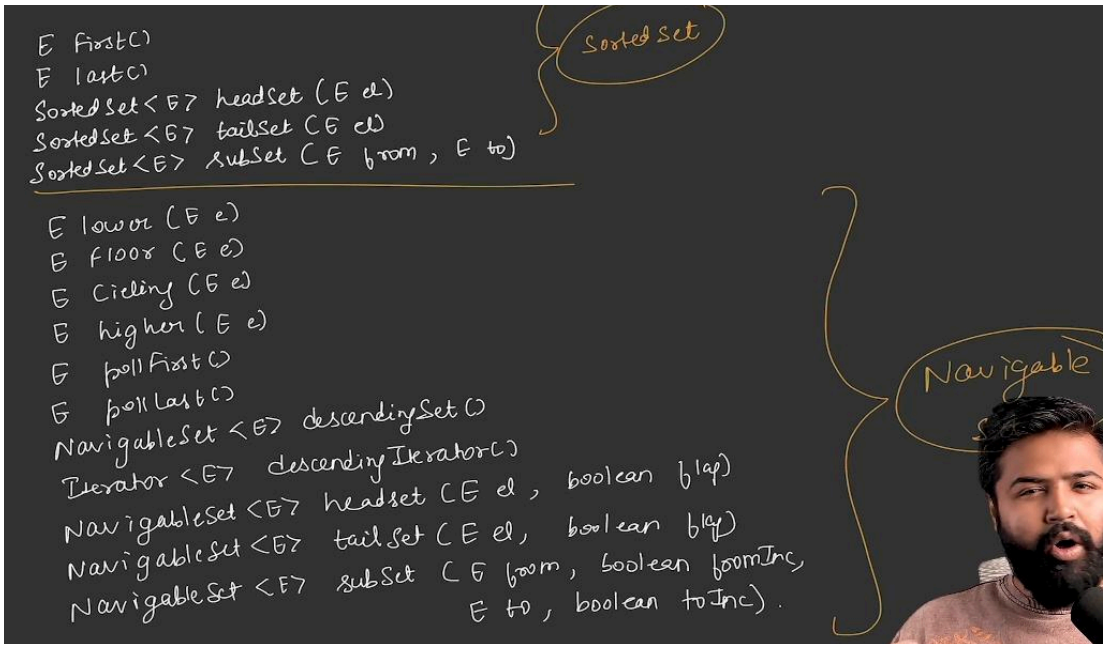


Sorted set & Navigable set are Interfaces.

Cunstroctors of Hash-Set & Linked-HashSet

```
// Constructors of HashSet ,& LinkedHashSet
            Set<Integer> set = new HashSet<>(); // internal size--> 16
        // initial capacity
            Set<Integer> set2 = new HashSet<>(100);
        // capacity, Load factor--> defaults---> 16, 0.75
            Set<Integer> set3 = new HashSet<>(100, 0.8f);
        // using another collection
            Set<Integer> set4 = new HashSet<>(List.of(1,2,3,4,5,6,7,8,9));
    // both do'nt have their methods because they implements collection and iterable interfaces.
```

Methods of TreeSet class




```
// Cunstroctors of TreeSet
        //Set<Integer> set2 = new TreeSet<>(List.of(2,5,7,9));
        TreeSet<Integer> set = new TreeSet<>();
            set.add(80 );
            set.add(23 );
            set.add(10 );
            set.add(90 );
            set.add(50 );
    // Sorted Set Interface's Methods
        System.out.println(set.first()); // to get the smallest value.--> 10
        System.out.println(set.last()); // to get the largest value.--> 90
```

```
        System.out.println(set.headSet(80 )); // it gives the smaller no's than 80, 80 is excluded.
        System.out.println(set.tailSet(80));// it gives the 80 and larger then 80.
        System.out.println(set.subSet(23, 80));//23 is inclusive & 80 is exclusive.
    // Navigable Set extends Sorted Set methods
        System.out.println(set.lower(80));// gives largest no lower than 80.
        System.out.println(set.floor(80));// gives greatest element less then or equal to 80.
        System.out.println(set.higher(80));// smallest no greater than 80.
        System.out.println(set.ceiling(80 ));// smallest no greater than or equal to 80.
        // System.out.println(set.pollFirst());// it gives & removes the smallest no.
        // System.out.println(set.pollLast());// it gives & removes the largest no.
        System.out.println(set.descendingSet());
        Iterator<Integer> it = set.descendingIterator();
            while(it.hasNext()){
                System.out.println(it.next());
            }
        System.out.println(set.headSet(80, true));// to include 80.
        System.out.println(set.tailSet(80, false));// to exclude 80.
        System.out.println(set.subSet(10, false, 80, true));
```

MAP interface methods

Map ko sabhi methods khud se define krne pdete ha kuki ye kisi bhi interface ko impliment nahi krte ha.

```
    Map<Integer, String> map = new HashMap<>();
  // RollNumber, Names
    map.put(101, "Priyansh");
    map.put(102, "Tanuja");
    map.put(103, "Tanveer");
    map.put(104, "Anuja");
    map.put(105, "Tanisha");
        System.out.println(map.size());
        System.out.println(map.isEmpty());
        System.out.println(map.containsKey(107 ));
        System.out.println(map.containsValue("Priyansh"));
        System.out.println(map.get(101));// gives value of the particular key.
        System.out.println(map.put(106, "Raghav"));// ans--> null,
        System.out.println(map.put(103, "Ramesha"));// ans-->Tanveer, but Tanveer is replaced by Ramesha.
        //System.out.println(map.remove(101));
        //map.putAll(map2);// kisi or map ki sabhi values ko put kr skte ha.
        //map.clear();
    //Important Methods of Map--->
        map.keySet();
        Set<Integer> set = map.keySet();
        System.out.println(set);// yha pr map ki sabhi keys ka set array bn jayga.
        map.values();
        Collection<String> c = map.values();
        System.out.println(c );// yha prr map ki sabhi values aa jaygi.
        map.entrySet();
        // Set<Map.Entry<Integer, String>> entries = map.entrySet();
        // System.out.println(entries);// yaha pr key & values dono mil jate ha.
        System.out.println(map.getOrDefault(109, "unknown"));// default value.
        System.out.println(map.putIfAbsent(103, "Abhay"));
        //putIfAbsent does not replace the existing value.
        map.remove(101, "Aditya");// it only removes if there is same match of key,value.
        map.replace(101, "Sonu");//agar key already exist krti hogi to hi value replace hogi.
        map.replace(101, "Tanujing", "RAngbaaz");// only replace agar old value or key match krte ha.
        // Set<Map.Entry<Integer, String>> entries = map.entrySet();
        //     for(Map.Entry<Integer, String> entry : entries){
        //         Integer key = entry.getKey();
        //         String value = entry.getValue();
        //             System.out.println(key + ", " + value);
        //     }
```

```
        // Immutable map
        Map<Integer, String> map2 = Map.of(202, "Ramesh", 203, "Arnav" );
        Set<Map.Entry<Integer, String >> entries = map2.entrySet();
            System.out.println(entries);
```

Cunstroctors of HashMap & Linked HashMap

```
// HashMap & LinkedHashMap--> both works same.
          // Cunstroctors
          Map<Integer, String> map = new HashMap<>();
          Map<Integer, String> map2 = new HashMap<>(100);
          Map<Integer, String> map3 = new HashMap<>(100,0.8f);
          Map<Integer, String> map4 = new HashMap<>(map3);
          Map<Integer, String> map5 = new HashMap<>(Map.of(101, "Aditya", 102, "Priyansh"));
```

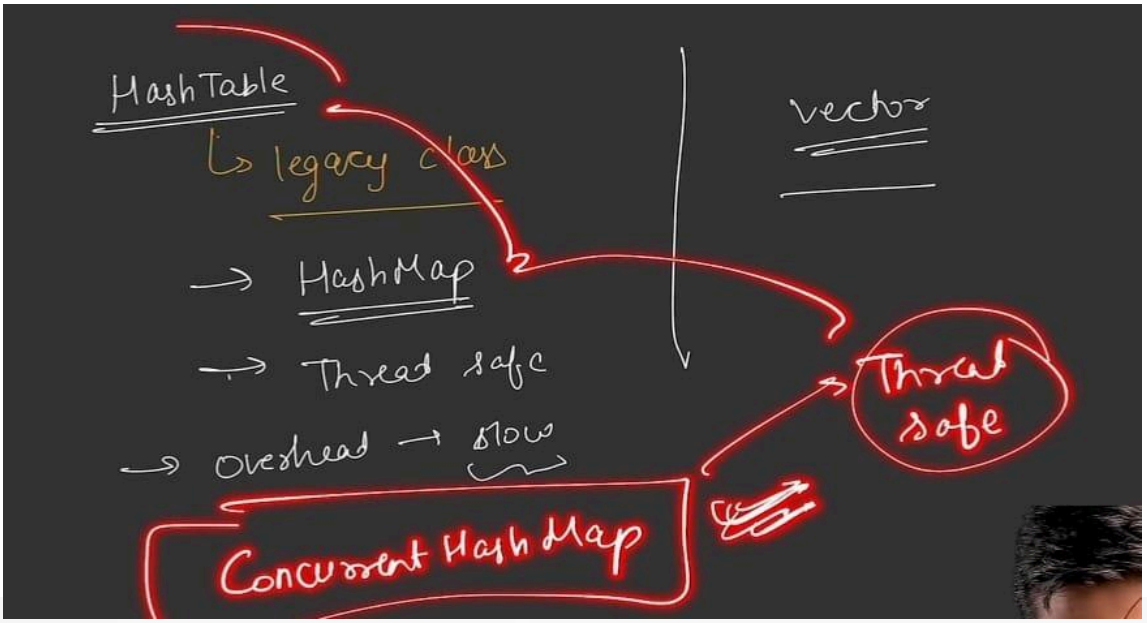
Methods of TreeMap

```
TreeMap<Integer, String> map = new TreeMap<>();
          map.put(101, "Aditya");
          map.put(102, "Rahul");
          map.put(103, "Ramlal");
          //Navigable Map Methods.
              System.out.println(map.firstKey());
              System.out.println(map.lastKey());
              System.out.println(map.firstEntry());
              System.out.println(map.lastEntry());
              System.out.println(map.headMap(103));
              System.out.println(map.tailMap(102));
              System.out.println(map.subMap(101, 103));
          //Sorted Map methods.
              System.out.println(map.lowerKey(102 ));
              System.out.println(map.lowerEntry(102 ));
              System.out.println(map.floorKey(102 ));
              // all alternatives of treeset are here for the key and entry.
```

Jo bhi methods treeSet me ha wo sabhi respectively treeMap ke key & entry ke liya bhi use hote ha.

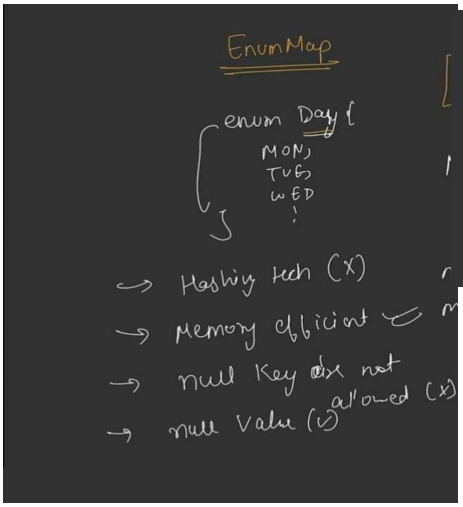
Other Map Classes

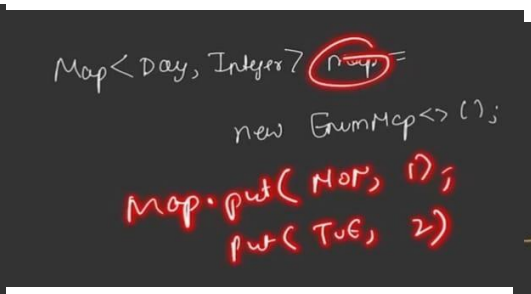
HashTables→

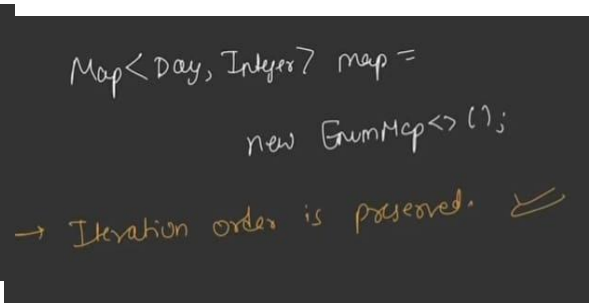


Properties→

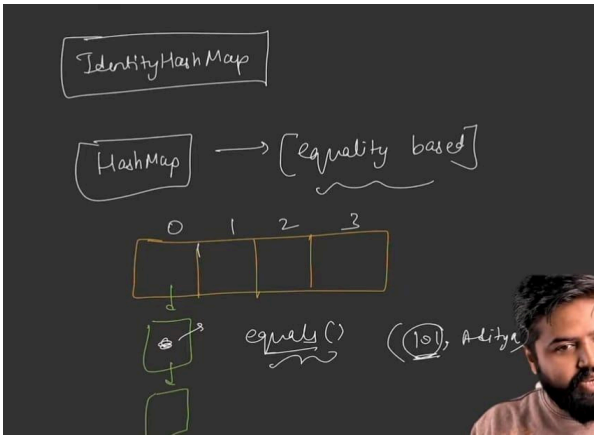
Enum Map→

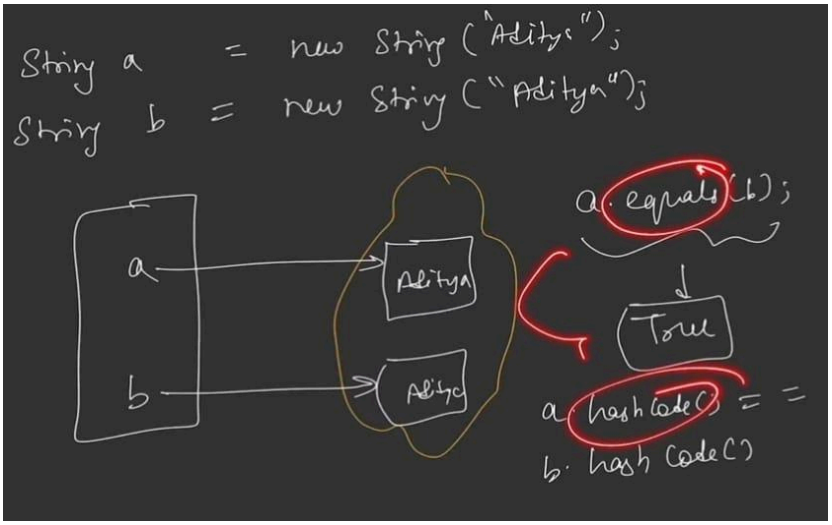






Identity HashMap→





Weak HashMap→